Project A

Attention Based Multiple Instance Learning and Fragility Guided Pooling

Whole Slide Image Classification

Course Number
ECE1512

Course Title
Digital Image Processing and Applications

Authors
Aaron Emmanuel and George Mikhaiel

Date Due
November 4, 2026

Abstract

Attention-based multiple instance learning (ABMIL) aggregates patch representations under slide-level supervision. We evaluate a fixed encoder, learned projection, gated attention operator and linear classifier on CAMELYON16 (C16), CAMELYON17 (C17) and BReAst Carcinoma Subtyping (BRACS) features. Thirty-epoch baselines selected by validation area under the receiver operating characteristic curve (AUROC) plus macro F1 achieved held-out accuracies of 88.37%, 78.50% and 46.51%, respectively. We formulate two proposals: sensitivity-controlled attention–mean pooling and normalized attention entropy regularization. The implemented pooling method uses the probability change under a deterministic suppression probe to set a slide-specific mixture without additional trainable parameters. Its full held-out accuracies are 86.05%, 78.50% and 33.72%, with macro F1 0.8480, 0.5218 and 0.2316. Macro F1 decreases relative to ABMIL on all three datasets. Fixed-mixture and pure-mean controls separate adaptive allocation from uniform smoothing. Gate diagnostics reveal that the top-fraction probe removes most attention mass on average. Class-specific errors qualify the aggregate comparisons. The study measures performance under a fixed feature and split protocol; single-seed evaluation, cross-platform training and patient dependence limit inference about superiority and clinical generalization.

1 Introduction and Problem Setting

Whole-slide classification requires learning from a large collection of tissue patches while supervision is often available only for the slide. Multiple instance learning (MIL) represents each slide as a bag and each patch representation as an instance, avoiding the assumption that every patch shares the slide diagnosis [1]. We study how learned attention combines fixed patch features, characterize its class-specific errors, and investigate whether sensitivity to evidence suppression can inform the balance between focused and distributed pooling.

Only the feature projection, attention aggregator and slide classifier are optimized. The upstream image encoder remains fixed. This separation permits analysis of aggregation while holding the supplied visual representation constant. The empirical evidence comprises full baseline, adaptive pooling, fixed-mixture and pure-mean comparisons on three datasets.

A positive slide may contain mostly ordinary tissue. Assigning the slide label to every patch would therefore introduce misleading instance supervision. MIL instead learns to aggregate patch evidence and trains against the slide label. The same feature vector can contribute differently depending on the competing evidence in its bag.

Patch softmax normalizes attention scores over valid instances, whereas class softmax normalizes the final logits over slide classes. These operations describe different distributions. Attention coefficients specify relative pooling weights; they are neither patch-level tumour probabilities nor an independently validated causal explanation.

2 Literature Review

Whole Slide Representation

A whole-slide image (WSI) is represented by N patch feature vectors of dimension D, with N determined by tissue content and patch extraction. An image encoder maps each patch to a numerical representation, an aggregator maps the variable-sized collection to a slide embedding, and a classifier predicts the slide class. Figure 1 illustrates a common configuration with a fixed pretrained encoder and trainable pooling and classification modules. The available supervision consists of slide labels rather than annotations for individual patches [1].

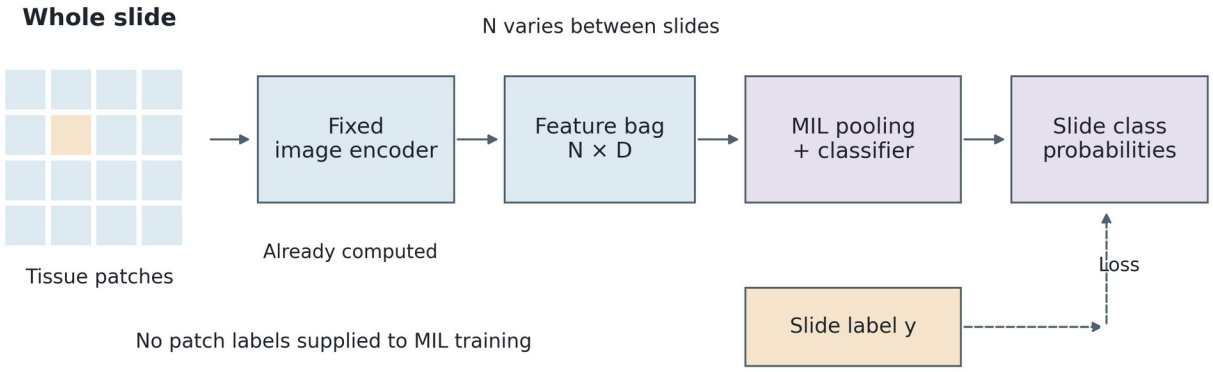


Figure 1. Whole-slide classification with a fixed feature encoder. Original schematic based on multiple instance learning [1]. N denotes the patch count and D the feature dimension. Purple modules are trainable; the dashed path represents slide-level supervision. The tile grid is illustrative rather than a histology image.

Weak Supervision

Classical binary MIL assumes that a positive bag contains at least one positive instance and a negative bag contains none [1]. This is a useful abstraction for sparse metastasis detection. It is less literal for multiclass subtyping: a slide label does not mean that every patch depicts that subtype. Treating all patches as independently labeled examples could therefore teach the model from incorrect targets. MIL instead learns which patch information helps predict the shared slide label.

The distribution of diagnostic evidence depends on the classification task. Metastasis detection may depend on a small tissue region, whereas multiclass categorization can depend on broader morphological patterns. The suitability of an aggregation operator is consequently related to both the prevalence of informative patches and the distinctions represented by the slide labels.

Attention Based Multiple Instance Learning

Pooling defines the information retained when patch representations are mapped to a slide embedding. Mean pooling assigns equal weight to every patch; elementwise max pooling retains the largest value in each feature dimension. Instance-based approaches instead aggregate patch predictions. An elementwise maximum of embedding coordinates need not correspond to a single observed patch. ABMIL introduces learned pooling weights [1]. Figure 2 isolates the effect of weighting through a scalar example.

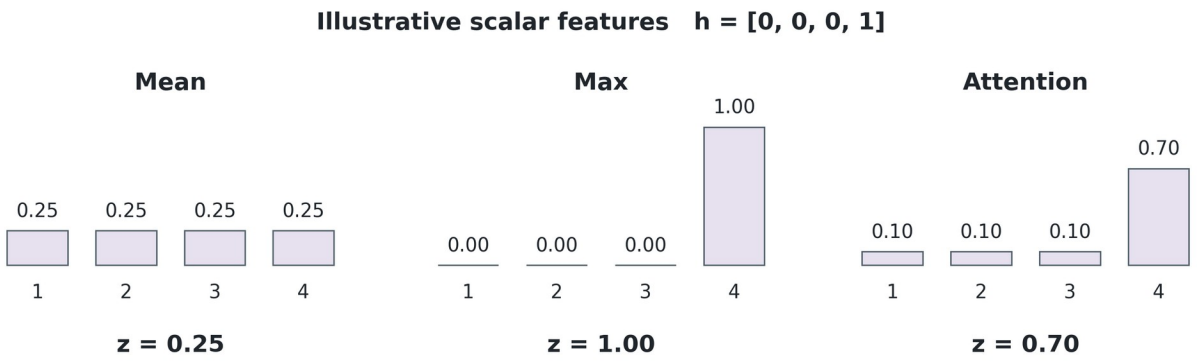


Figure 2. Original arithmetic example of pooling. Three scalar features are zero and one is one. Mean, scalar max and an illustrative attention distribution produce 0.25, 1.00 and 0.70. Attention weights are chosen, not learned or measured here. The example illustrates pooling [1]; it is not a classification result or a plot of tissue probabilities.

Weighted Aggregation

Let h_i denote the projected embedding of patch i and a_i its pooling weight. The slide embedding is $z = \Sigma_i a_i h_i$, with nonnegative weights summing to one. In Figure 2, the fourth patch contributes 0.70×1 and the others contribute zero. Equal weighting dilutes that isolated feature; max retains it completely. If the fourth value represented an artefact rather than useful evidence, the same arithmetic would favor the artefact. Thus the example explains a trade-off, not a universal ranking of pooling methods.

Ilse et al. [1] learn attention weights from instance embeddings. Their gated formulation combines hyperbolic tangent and sigmoid branches before scalar scoring and softmax normalization. The weighted embedding enters a bag classifier, and a bag-label likelihood supplies the learning signal. Variable bag sizes are accommodated through normalized aggregation. This embedding-based formulation separates the feature representation, attention operator and final classification function.

Interpretation of Attention Weights

An attention weight is a relative coefficient in the pooled slide representation, rather than a calibrated probability of patch malignancy. The class prediction also depends on the patch embedding and classifier parameters. A normal patch can therefore receive high attention in a correctly classified normal slide. Softmax normalization further couples the coefficients: adding instances can change a patch weight without changing its unnormalized score. Attention overlays characterize the aggregation behavior, but do not independently establish localization accuracy or explanation faithfulness.

Instance Supervision and Critical Instance Aggregation

ABMIL learns attention through the slide-level classification objective. Clustering-constrained attention multiple instance learning (CLAM) and dual-stream multiple instance learning (DSMIL) introduce additional structure through instance supervision and critical-instance aggregation, respectively [2, 3]. Figure 3 summarizes the distinction between these mechanisms.

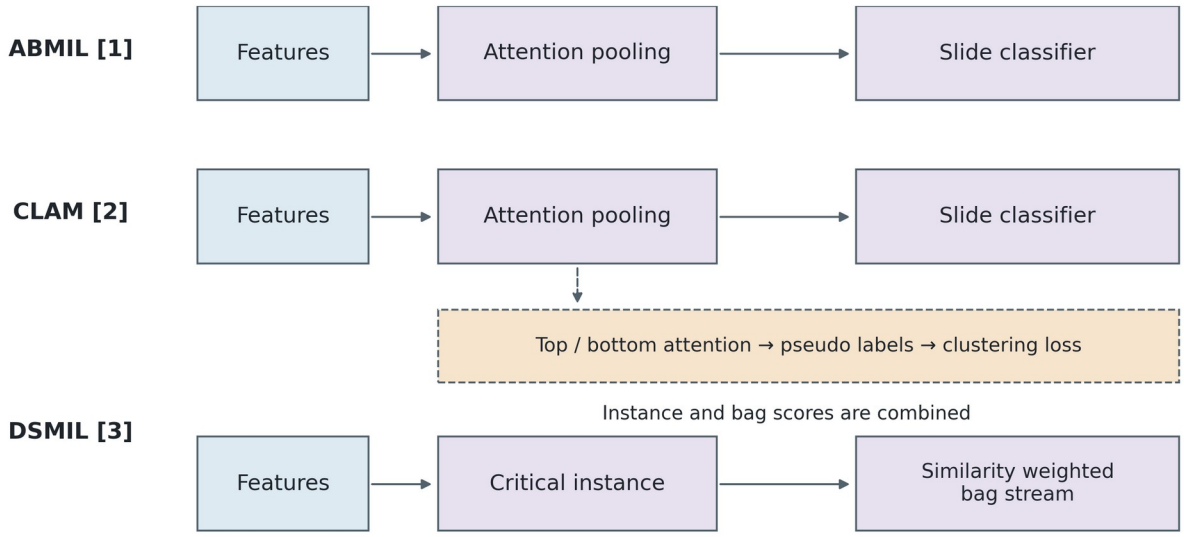


Figure 3. Original simplified comparison of ABMIL [1], CLAM [2] and DSMIL [3]. CLAM adds training supervision from attention-derived pseudo labels; DSMIL uses a critical instance to guide its bag stream and combines instance and bag scores. The orange dashed branch is an auxiliary training objective. Feature encoders and class-specific implementation details are omitted.

CLAM and Attention Derived Pseudo Labels

Lu et al. [2] combine slide classification with an instance clustering objective in CLAM. Highly and weakly attended patches supply pseudo labels for a smooth support vector machine (SVM) loss, alongside slide cross-entropy. Their renal, lung and lymph-node studies include patient-grouped partitions and independent cohorts. The attraction is additional learning structure without manually labeling every patch. The limitation is that predicted importance becomes a training target: incorrect attention can generate incorrect pseudo labels, and the validity of subtype exclusivity depends on the classification task.

An auxiliary instance loss changes the supervision received by the feature projection as well as the optimization objective. Its contribution is distinct from the effect of increasing model capacity. Attribution of a performance difference to pseudo-label supervision therefore depends on separating the objective from simultaneous architectural changes; slide-level classification performance alone does not isolate these effects.

DSMIL and the Critical Instance

Li et al. [3] use an instance stream to select a highest-scoring critical patch and a second stream to aggregate features by similarity to it. The slide prediction combines the two scores. Their pipeline also includes contrastive feature learning and multiscale fusion; evaluation covers C16 and The Cancer Genome Atlas (TCGA) lung cancer, with free-response receiver operating characteristic (FROC) curves for localization. A critical patch can anchor supporting evidence, but an incorrect anchor can redirect aggregation. Feature learning, multiscale processing and the aggregation operator constitute distinct components of the complete pipeline.

CLAM and DSMIL illustrate two different uses of weak supervision. CLAM derives instance-level training targets from attention, whereas DSMIL organizes aggregation around a reference instance. Both impose additional inductive structure on the relationship between local evidence and the bag prediction, with different sensitivities to errors in the evidence selected by the model.

Contextual Aggregation and Feature Representation

Transformer Based Aggregation

Transformer-based multiple instance learning (TransMIL), introduced by Shao et al. [4], models correlations between patches. Two transformer layers surround a pyramid position encoding generator (PPEG); a Nyström approximation addresses long sequences. Training uses slide cross-entropy, with evaluation on C16 and TCGA lung and renal subtyping. Figure 4 contrasts contextual token processing with independent initial scoring in ABMIL. An instance representation can depend on other instances before the final slide readout.

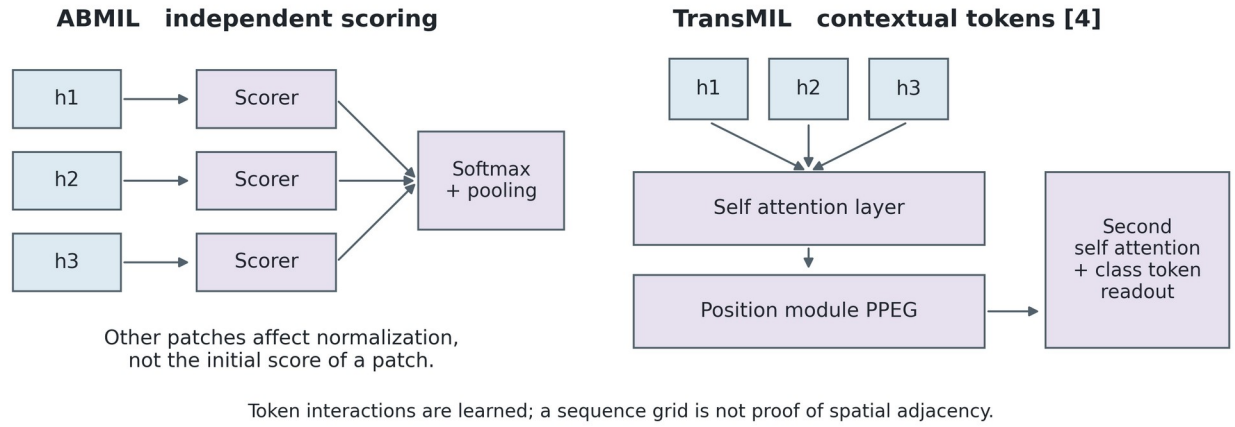


Figure 4. Original conceptual comparison based on ABMIL [1] and TransMIL [4]. Arrows on the right summarize token interaction, not a complete transformer computation graph. PPEG is a position module within the sequence pipeline. A class token participates in the transformer processing and supplies the final readout; it is omitted from the input row for clarity.

Two bags can contain an identical patch embedding but different surrounding tissue. An independent scorer assigns that embedding the same initial score, although softmax can change its final weight. A contextual model can additionally alter its representation through interactions with other tokens. This distinction concerns expressive capacity; it does not establish that additional capacity necessarily improves generalization in small cohorts.

Positional encoding introduces assumptions about the organization of patch features. A square grid constructed from an ordered sequence does not necessarily preserve the physical adjacency of tissue. Its spatial interpretation depends on the coordinate mapping, ordering and padding convention. Computational comparisons also depend on bag sampling and truncation, because these operations change the evidence available to the aggregator as well as its resource requirements.

Pseudo Bag Learning

Double Tier Feature Distillation

Double-tier feature distillation multiple instance learning (DTFD-MIL), proposed by Zhang et al. [5], randomly partitions a slide into pseudo-bags that inherit its label. A first MIL tier learns from these groups and distils features; a second tier predicts the slide from the distilled information. Both tiers have classification objectives. The study evaluates C16 and TCGA lung cancer. Figure 5 illustrates the two levels while leaving the particular distillation strategy abstract.

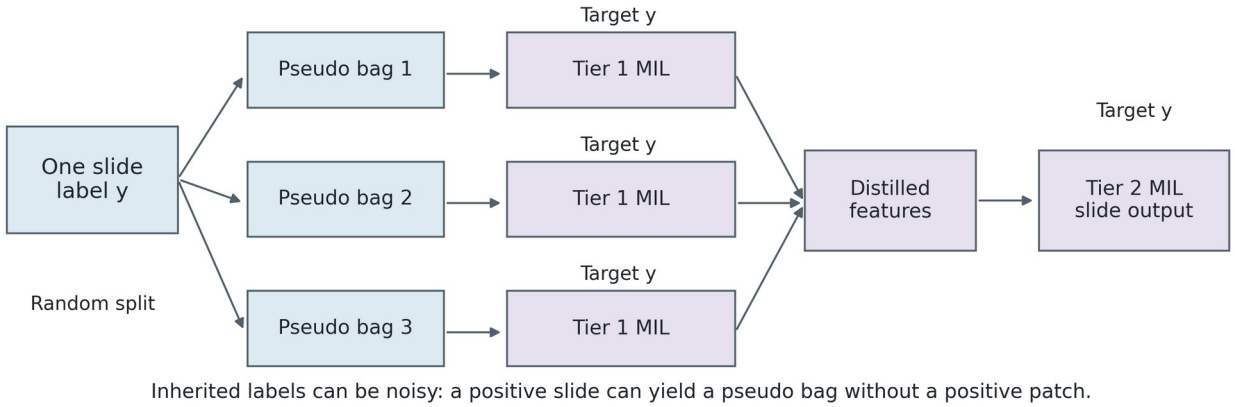


Figure 5. Original simplified DTFD-MIL schematic based on [5]. Each pseudo-bag inherits the parent slide label for first-tier training; distilled features then support a second-tier slide prediction. The number of groups is illustrative. The diagram does not imply that the groups have independent patient identities or verified patch labels.

Label Noise Under Inherited Supervision

In a simplified positive slide containing twelve patches and exactly one diagnostic patch, partitioning into three nonempty groups places the diagnostic patch in only one group. All groups inherit the positive slide label, so two are positive-labeled groups without the positive patch. This arithmetic example illustrates label noise rather than estimating its prevalence in a dataset. DTFD-MIL explicitly recognizes the corresponding risk of positive pseudo-bag labels [5].

The number of pseudo-bags controls the amount of evidence available within each training group. Smaller groups increase the number of optimization units per slide but may exclude diagnostic regions or separate contextual information. These groups remain dependent observations drawn from the same patient. Their increased count therefore changes the training distribution without increasing the number of independently observed disease cases.

Implications for Generalization

Pseudo-bag construction changes the distribution of training inputs and inherited labels, whereas parameter or attention regularization constrains the learned model. These mechanisms address different aspects of limited supervision. Performance measured on pseudo-bags also differs in statistical unit from performance measured on complete slides, even when the groups inherit the same target label.

Repeated partitioning changes the gradients observed during optimization while preserving the original patient population. This distinction between optimization diversity and independent sample size limits an interpretation of pseudo-bag learning as an increase in labeled clinical data.

Attention Concentration and Regularization

Attention concentration describes the allocation of pooling mass to a small subset of instances. It is a possible contributor to limited generalization, although loss trajectories alone do not identify that mechanism. Masked hard instance mining multiple instance learning (MHIM-MIL) and attention-challenging multiple instance learning (ACMIL) address reliance on salient instances through different training objectives and masking procedures [6, 7], summarized in Figure 6.

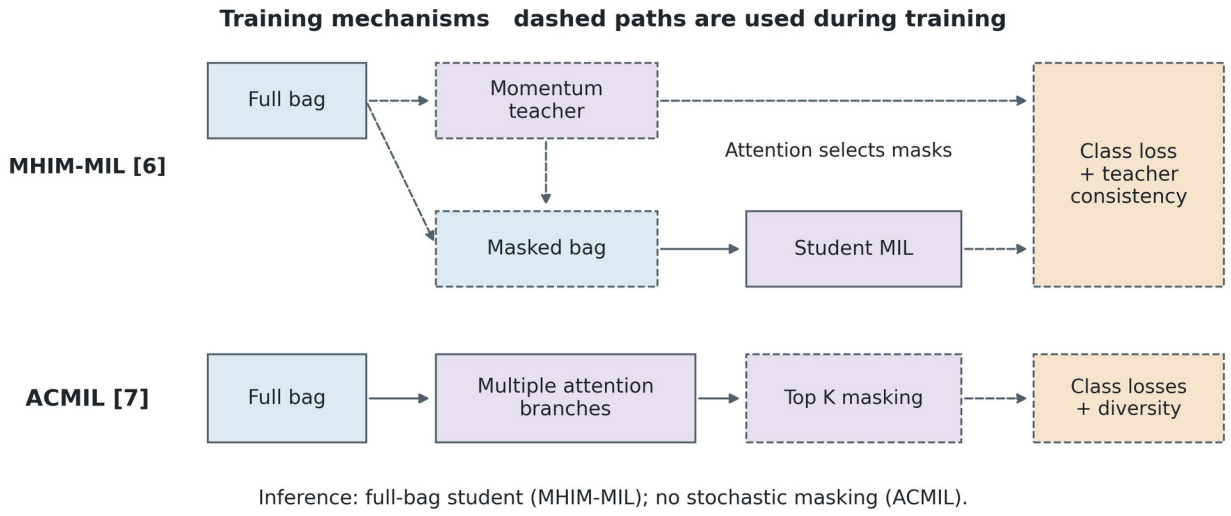


Figure 6. Original training schematic based on MHIM-MIL [6] and ACMIL [7]. MHIM-MIL uses teacher attention to guide masking and a teacher prediction for consistency. The teacher follows an exponential moving average of student parameters, an update omitted here. Auxiliary branch heads and detailed loss connections are omitted. Both methods remove masking for inference.

Teacher Guided Hard Instance Mining

Tang et al. [6] combine a slide classification loss with consistency to a momentum teacher. Teacher attention guides which salient instances the student does not see during training; full-bag student inference follows.

Experiments use C16 and TCGA lung cancer. The aim is to learn from harder remaining evidence. The teacher update, masking strategy and consistency weight increase the number of hyperparameters governing optimization. If the removed region contains the only diagnostic signal, masking can also remove evidence needed to preserve the slide label.

Attention Diversification

ACMIL [7] combines multiple attention branches, branch classification and diversity objectives, and stochastic masking of selected high-attention weights. Masking is disabled at inference. Evaluation uses two pretrained backbones, C16, BRACS in a three-class setting, and a private cytology dataset. The method establishes attention diversification as an existing approach to generalization. Its localization analysis also indicates that broader attention does not necessarily identify lesions more precisely.

Normalized attention entropy, $H(a)/\log N$ for $N > 1$, where $H(a) = -\Sigma_i a_i \log a_i$, quantifies dispersion relative to a uniform distribution. The cumulative mass assigned to the highest-weight instances provides a complementary concentration measure. Uniform attention maximizes entropy but can retain irrelevant tissue. Concentration, classification performance and localization accuracy consequently describe different properties of a model; none is interchangeable with the others.

Pretrained Feature Representations

Chen et al. [8] introduce UNI, a general-purpose self-supervised pathology encoder. Its role is to learn transferable image representations rather than define a particular MIL pooling rule. Holding the encoder fixed isolates differences in aggregation and training, whereas encoder replacement introduces a separate source of performance variation. Matching feature dimensionality alone does not establish equivalence between encoders.

Comparison of Learning Signals

Table 1. Learning signals and aggregation mechanisms. The limitations concern supervision, representation and evaluation rather than a universal performance ordering. All listed MIL methods can train without manual patch labels; UNI provides encoder context.

Method	Learning signal and aggregation	Main rationale	Methodological limitation
ABMIL [1]	Slide-label likelihood; learned weighted pooling	Compact variable-bag reference	No explicit pairwise contextualization
CLAM [2]	Slide cross-entropy plus instance clustering	Extra weak supervision	Pseudo-label errors and task assumptions
DSMIL [3]	Critical-instance and bag streams	Strong cue plus supporting evidence	Sensitivity to the critical instance
TransMIL [4]	Slide cross-entropy; contextual self-attention	Models patch relationships	Resource cost and spatial-order assumptions
DTFD-MIL [5]	Two tiers; pseudo-bag classification	More training groups	Noisy labels; no new independent patients
MHIM-MIL [6]	Classification plus teacher consistency	Learns beyond salient instances	Teacher and masking choices
ACMIL [7]	Branch diversity and training-time masking	Diversifies attended evidence	Broad attention can reduce localization precision
UNI [8]	Self-supervised encoder pretraining	Improves input representations	Encoder changes confound pooling comparisons

Comparability of Published Results

Published performance estimates are conditional on the evaluation protocol. MHIM-MIL uses repeated three-fold C16 evaluation [6], whereas ACMIL retains the official test partition and reports five initializations [7]. ACMIL evaluates BRACS in a three-class setting. Label definitions, encoders and training objectives vary across studies, so numerical results from different papers are not measurements from a common controlled experiment.

Attribution of a performance difference to aggregation depends on the feature representation, label mapping, data partition and checkpoint-selection criterion. Variation in any of these factors changes the comparison. Selection based on held-out outcomes introduces dependence between model development and the test estimate, while variation across random initializations contributes uncertainty that a single-seed result does not quantify.

Slide independence differs from patient independence because sections from the same patient can share biological and acquisition characteristics. Patient-grouped partitions, such as those used in the CLAM study [2], separate this dependence from generalization to unseen cases. Increasing the number of patches or pseudo-bags does not increase the number of independent patients.

Synthesis and Interpretation

Classification Metrics

Accuracy is the fraction of slides whose predicted class is correct and can be dominated by frequent classes. F1 is the harmonic mean of precision and recall; macro F1 averages the class-specific values equally. AUROC, also termed AUC, measures ranking over thresholds. Macro AUROC averages one-versus-rest class AUROCs. Cross-entropy (CE) also responds to confidence: confidently incorrect predictions can increase loss without a change in the number of errors. These measures therefore characterize complementary properties of slide classification.

In an illustrative test set with 90 class-A slides and 10 class-B slides, assigning every slide to A yields 90% accuracy but zero F1 for class B and a macro F1 of approximately 0.474. This constructed example explains how aggregate correctness can conceal class-specific failure. Accuracy, macro AUC, macro F1 and loss histories consequently describe complementary aspects of slide-level classification.

Causal Interpretation of Generalization

A training–validation loss gap can indicate limited generalization, but it does not establish attention concentration as the underlying cause. Feature quality, model capacity, weak-label noise and dependence between samples offer different explanations. Similarly, an association between attention entropy and classification performance does not identify whether entropy itself is causally responsible. These distinctions limit mechanistic conclusions derived from learning curves or aggregate scores alone.

Aggregation Bias and Generalization

Mean and attention pooling impose different biases on the slide representation. A mean component retains information distributed across the bag, while attention can emphasize a localized diagnostic cue. Their combination can preserve both forms of evidence, but can also introduce irrelevant tissue into the representation. The effect depends on the prevalence and specificity of informative instances, rather than on a general preference for broader aggregation.

Attention regularization introduces a related trade-off. A less concentrated distribution can reduce reliance on a few instances, yet it can suppress the distinction between sparse diagnostic regions and abundant background tissue. MHIM-MIL and ACMIL [6, 7] address this problem through altered training exposure and branch diversity. Their mechanisms differ from directly constraining entropy, even when the resulting attention distributions share similar summary statistics.

Architectural expressiveness, regularization and representation quality provide distinct explanations of model behavior. Classification metrics quantify slide-level outcomes, while attention maps describe the allocation of pooling weights. Agreement between high-attention regions and visible tissue morphology does not independently

establish causal faithfulness or calibrated lesion localization. The interpretation of each result is therefore limited to the outcome and statistical unit evaluated.

3 Datasets and Feature Preparation

C16 and C17 concern breast-cancer metastases in lymph-node slides [9, 10]. The feature artifacts define binary slide classification for C16 and four slide classes for C17. The original C17 challenge also includes patient-level staging, which is outside the outcome evaluated here. BRACS comprises normal tissue and six lesion categories: pathological benign lesions, usual ductal hyperplasia, flat epithelial atypia, atypical ductal hyperplasia, ductal carcinoma in situ and invasive carcinoma [11]. Numerical class identifiers follow the supplied artifacts; no clinical-name mapping is inferred from their order.

Table 2. Feature datasets used in the experiments

Dataset	Classes	Train	Val	Test	Patches per slide
C16	2	242	27	129	153 to 55,917
C17	4	269	30	200	447 to 36,126
BRACS	7	389	62	86	155 to 30,340

Each patch has a 384-dimensional representation. The available feature files contain 398 C16, 499 C17 and 537 BRACS slides; these counts describe the analyzed artifacts rather than the complete original cohorts. Features were obtained from the LeonYe repository [12]. All three files matched the published Secure Hash Algorithm 256-bit (SHA-256) fingerprints, and full scans found no nonfinite feature values. No additional slide exclusion, patch subsampling or feature normalization was applied. The exact upstream encoder architecture, pretraining cohort and patch extraction settings are not established by the stored feature dimension alone and are not inferred here.

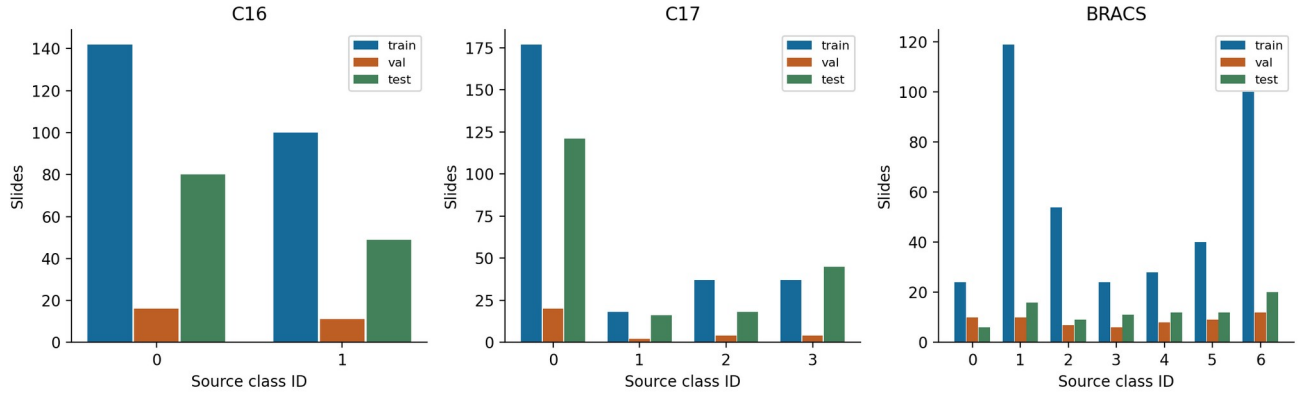


Figure 7. Actual class counts from the preflight audits. Validation and test class proportions differ from training, particularly for C17 and BRACS. Numeric IDs are retained to avoid unsupported clinical labels.

Split Construction

The loader enumerates slide groups in Hierarchical Data Format version 5 (HDF5) files. C16 slide identifiers containing test define the test set; a seeded, label-stratified 10% subset of the remaining slides defines validation. C17 centres with indices of at least 3 define the test set, with a seeded, stratified 10% slide-level validation subset from the other centres. BRACS uses the supplied comma-separated values (CSV) split assignments. Few-shot subsampling is disabled.

Slide identifiers are disjoint across the saved partitions. C17 nevertheless has 27 patients represented in both training and validation, while test patients are separate from both. Its validation results are consequently not patient-independent. The original partitions are retained to preserve comparability. Slide-level disjointness alone does not establish patient independence for C16 or BRACS.

4 Baseline Architecture and Optimization

Input Representation and Masks

The dataset loader reads one slide feature matrix as 32-bit floating-point values, together with its label and optional patch coordinates. Matrices are required to be two-dimensional and nonempty. Lazy loading avoids keeping the full feature archive in memory. Coordinates are retained as metadata but do not enter the ABMIL scoring function.

The collation function pads variable-sized bags to the maximum length in a batch and returns a Boolean validity mask. With batch size B = 1, each input has shape [B, N, 384], where N is the slide patch count. Labels are integer class identifiers. Invalid positions are excluded from attention normalization and pooling; zero padding alone would not eliminate contributions induced by learned biases.

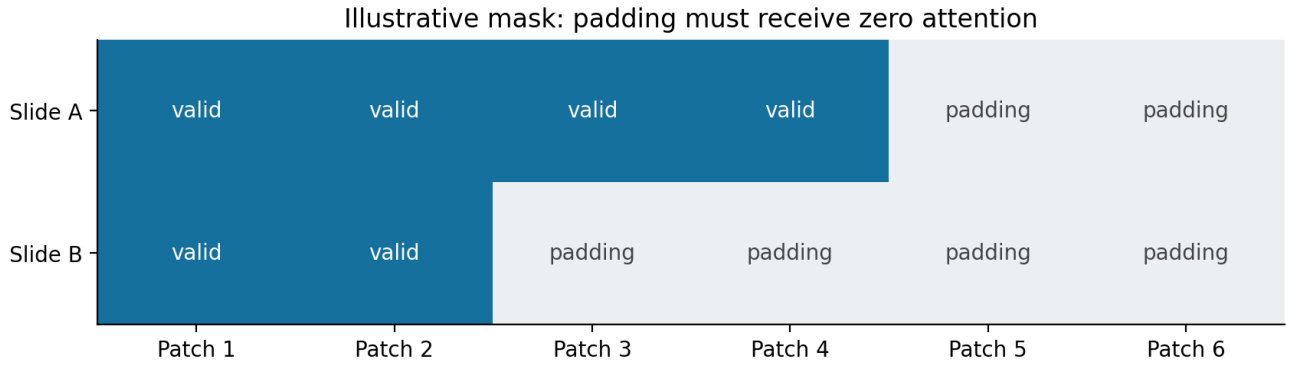


Figure 8. Illustrative two-slide batch, not an experimental observation. The model must exclude padded entries even though they occupy positions in the tensor. A learned linear bias means a zero input alone is not enough to guarantee a zero pooling contribution.

Table 3. Tensor shapes through ABMIL

Code value	Shape	Interpretation
input / x	[B, N, 384]	Patch features, including padded positions if needed.
mask	[B, N]	True for valid patches.
med_feat	[B, N, 256]	Projected patch representations.
a_logits	[B, 1, N]	One unnormalized attention score per patch.
attention	[B, 1, N]	Softmax weights; padded entries have weight zero.
bag_feat	[B, 256]	Weighted slide representation.
logits	[B, C]	Class scores for each slide.
labels	[B]	Integer ground-truth slide class.

Feature Projection and Classification

A learned linear map projects each 384-dimensional patch vector to 256 dimensions, followed by a rectified linear unit (ReLU). This adapts the supplied features without modifying the upstream encoder. A final linear map transforms the pooled 256-dimensional slide vector into C class logits, where C is 2, 4 or 7. Dropout is disabled.

The classifier therefore learns class boundaries in the aggregated representation, while the projection and attention operator determine the information available to those boundaries.

Gated Attention

Two learned 256-to-128 transformations process each projected patch. One branch uses the hyperbolic tangent and the other a sigmoid; their elementwise product enters a scalar scoring layer. Parameters are shared across patches and slides. A softmax over valid instances produces nonnegative weights summing to one.

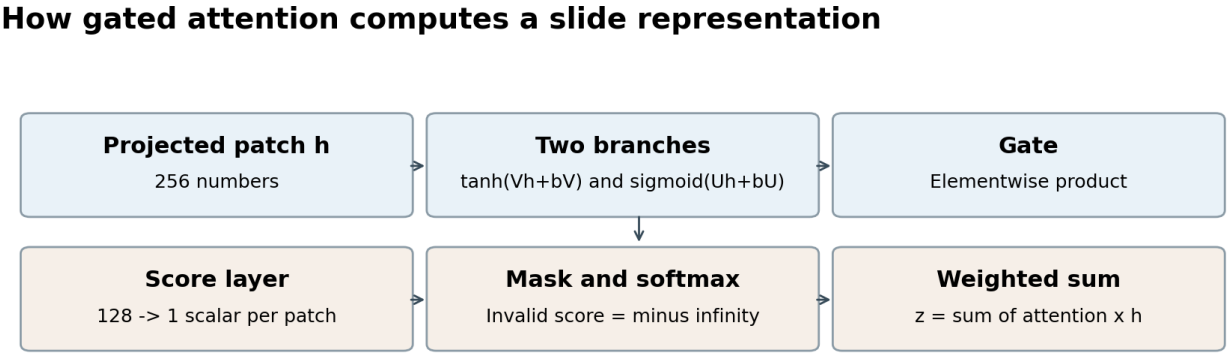


Figure 9. Gated attention combines two nonlinear branches before scalar scoring and patch normalization. The sigmoid within this scorer differs from the slide-level mixing coefficient defined for the pooling extension.

$$h_i = \mathrm{ReLU}\left(W_p x_i + b_p\right)$$

$$s_i = w_s^{T}\left[\tanh\left(V h_i + b_V\right) \odot \mathrm{sigmoid}\left(U h_i + b_U\right)\right] + b_s$$

$$a_i = \mathrm{softmax}_i\left(s\right), \qquad z = \sum_i a_i h_i, \qquad \ell = W_c z + b_c$$

Invalid scores are replaced by negative infinity before patch softmax, giving padded positions zero weight. Every slide must contain at least one valid patch. The normalized weights then define a weighted sum of the projected embeddings.

Aggregation and Component Contributions

The attention tensor [B, 1, N] multiplies the projected features [B, N, 256], yielding one [B, 256] slide representation after removal of the singleton dimension. The classifier returns [B, C] logits. Optional attention outputs permit inspection of the pooling distribution. The encoder supplies local visual features, the projection adapts them, attention allocates relative contributions, and the classifier converts the resulting slide vector into class scores.

Reordering valid patches changes neither the normalized weighted sum nor the prediction, apart from normal floating-point effects. No ordered spatial structure is encoded by this pooling operation. The score for each patch is computed from that patch, while the softmax denominator makes its relative weight depend on the other patches. High attention can show which features contributed strongly to pooling; it does not validate lesion localization.

Table 4. Trainable parameter counts derived from layer dimensions

Component	Parameters
Projection: 384 x 256 + 256	98,560
Two attention branches: 2 x (256 x 128 + 128)	65,792
Attention score: 128 x 1 + 1	129
Classifier: 256 x C + C	257 x C
Total C16 / C17 / BRACS	164,995 / 165,509 / 166,280

The fixed-mixture and fragility-guided models reuse these trainable layers. Their pooling operations add no learned parameters. Counts exclude the upstream frozen encoder, which is absent from the MIL optimization loop. These architectural roles explain the computation but do not quantify each component's causal contribution to performance; no encoder-replacement or projection-removal ablation is available.

Training and Checkpoint Selection

Optimization Objective

Each training iteration processes one slide, clears accumulated gradients, computes class logits and minimizes slide-level cross-entropy. Backpropagation updates the projection, attention and classifier jointly using AdamW, an adaptive moment optimizer with decoupled weight decay. Computation uses a central processing unit (CPU). Nonfinite training losses raise an error. Epoch loss is averaged across slides and accuracy uses the largest class logit.

Cross-entropy receives logits and integer labels, incorporating log-softmax internally. For a slide with true class y, its contribution is $-\log p_y$. Confident errors therefore receive a larger penalty, explaining how validation loss can rise substantially while accuracy changes only modestly.

Validation and Held Out Evaluation

Evaluation disables gradient tracking and sets model layers to evaluation mode. Class probabilities and labels are collected without parameter updates; mean cross-entropy is computed over slides. Splits lacking a configured class are rejected because macro AUROC would be undefined. Predictions use argmax without a fitted probability threshold.

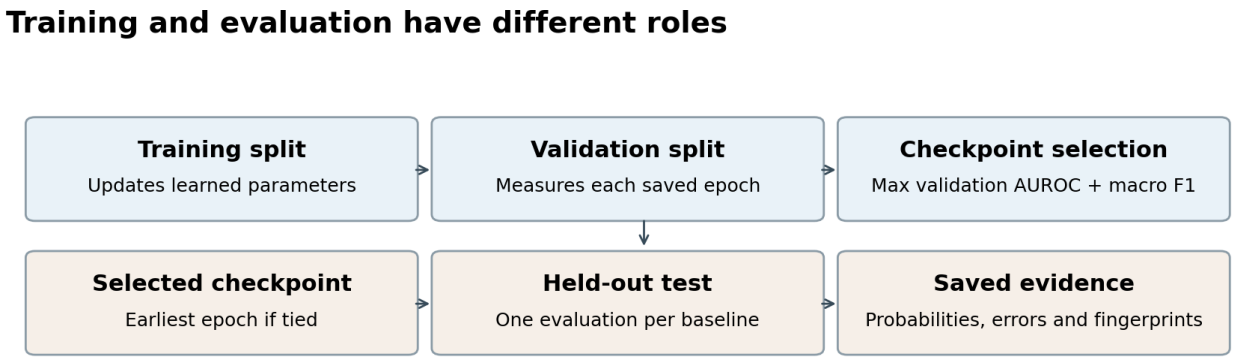


Figure 10. Validation selects the checkpoint; held-out test measures that already-selected checkpoint. All 30 epochs are trained even if the best checkpoint occurs early. The selected epoch is not necessarily the epoch with minimum validation loss.

Table 5. Baseline hyperparameters

Setting	Actual value
Input / projection / attention dimensions	384 / 256 / 128
Optimizer	AdamW
Learning rate / weight decay	0.0001 / 0.00001
Epochs / seed / batch size	30 / 42 / 1
Dropout / workers / CPU threads	0 / 0 / 2 per process
Loss / scheduler	Slide cross-entropy / none
Warmup	None
Checkpoint criterion	Maximum validation macro AUROC + macro F1; earliest tie.
Training environment	Python 3.9.5; PyTorch 2.8.0+cpu; Intel i7-1065G7 CPU.

Experiment Records and Selection Rule

The training runner fixes random seeds, constructs the saved partitions and loaders, and initializes the configured model. New output directories prevent silent replacement of earlier experiments. Each run records its configuration, dependency versions, command, split identifiers and source fingerprints before optimization.

After each epoch, the runner evaluates validation AUROC and macro F1, adds them and compares the sum with the best prior score. A strict greater-than comparison replaces the checkpoint, so an exact tie retains the earlier epoch. The runner records all 30 history rows and plots training and validation loss. The separate evaluate_run.py workflow was used for the final baseline tests, preserving per-slide probabilities and the selected checkpoint fingerprint.

Evaluation Safeguards

The evaluator checks that a full 30-epoch run has completed with consecutive history entries, recomputes the selected epoch from validation scores and checks the saved source hashes against the current training modules. It verifies disjoint slide IDs and regenerates the exact original splits. It then strictly loads best_state.pt, evaluates that checkpoint and records an evaluation plan, metrics, predictions, confusion matrix and per-class results. Checkpoint hashes ensure evaluation does not alter the weights. The exact evaluator source is archived with each test result.

The saved Torchmetrics results are cross-checked against scikit-learn. Accuracy is reported as a percentage; AUROC and F1 lie on a zero-to-one scale. Macro F1 gives each class equal weight, and macro AUROC averages one-versus-rest ranking performance. The latter need not imply strong recall at the argmax decision rule.

$$F1_c = \frac{2\ TP}{2\ TP\ +\ FP\ +\ FN}, \qquad macro\ \ F1\ = \frac{1}{C}\sum_c F1_c$$

Here TP, FP and FN denote true positives, false positives and false negatives for a class. Precision is TP/(TP + FP), and recall is TP/(TP + FN). AUROC is computed from the corresponding one-versus-rest probability scores.

Per-class F1 follows the saved zero-division convention, assigning zero when its denominator is zero. The evaluator also prevents duplicate evaluation records and output replacement. These safeguards reduce accidental reuse but cannot make previously observed test outcomes unknown. Earlier debugging and subsequent method development are disclosed with the reproducibility record.

5 Baseline Results and Interpretation

Each baseline result uses the checkpoint selected from its full 30-epoch run, with one slide as the evaluation unit. Tables and plots are derived from the saved class probabilities and labels. All three runs use seed 42. Differences across these tasks reflect their label spaces, sampled cohorts and learned representations; they are not a controlled measure of relative dataset difficulty.

Table 6. Held-out baseline performance

Dataset	Test N	Best epoch	Accuracy %	Macro AUROC	Macro F1	CE loss
C16	129	3	88.37	0.9224	0.8714	0.2700
C17	200	21	78.50	0.8226	0.5520	0.7969
BRACS	86	3	46.51	0.7855	0.3716	1.4110

Table 7. Selected validation results and change on test

Dataset	Val accuracy %	Val AUC	Val F1	F1 change on test
C16	100.00	1.0000	1.0000	-0.1286
C17	90.00	0.8854	0.7856	-0.2336
BRACS	56.45	0.8068	0.4429	-0.0712

C16 shows the strongest aggregate results in these runs. C17 accuracy is much higher than its macro F1, indicating uneven class performance. BRACS has the lowest macro F1 and broad confusion outside one well-recognized class. Macro averaging makes the weaknesses of less frequent classes visible rather than weighting every class by its number of test slides.

The descriptive test majority-class accuracies are 62.02% for C16, 60.50% for C17 and 23.26% for BRACS. ABMIL exceeds these references, but these numbers are computed from test label frequencies; they are not a separately trained or validation-selected baseline. Exceeding them alone does not establish satisfactory class coverage or clinical utility.

Learning Curves and Generalization

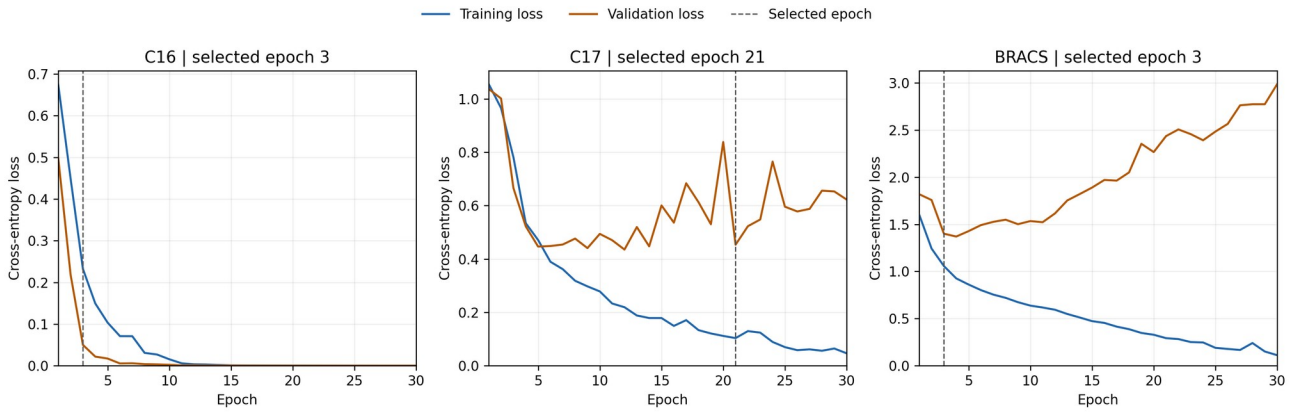


Figure 11. Training and validation cross-entropy across all 30 epochs. Each panel has its own vertical scale. Dashed lines mark selection by validation AUROC plus macro F1, not selection by minimum loss.

Table 8. Loss diagnostics

Dataset	Selected epoch	Selected val CE	Final train CE	Final val CE	Minimum val CE
C16	3	0.0499	5.994e-05	1.836e-05	1.836e-05
C17	21	0.4533	0.0464	0.6225	0.4352
BRACS	3	1.4000	0.1116	2.9865	1.3701

For C16, validation AUROC and F1 first reach 1.0 at epoch 3. Later epochs may reduce cross-entropy further, but the selection sum cannot exceed its maximum of 2. The strict tie rule therefore keeps epoch 3. Perfect validation on only 27 slides does not imply perfect performance outside that subset; the test result is 88.37% accuracy.

C17 continues improving beyond the initial debug epochs and selects epoch 21. By the final epoch, training loss is 0.0464 while validation loss is 0.6225. This divergence is consistent with overfitting, although it does not prove its biological cause. BRACS selects epoch 3; final training loss falls to 0.1116 while validation loss rises to 2.9865. That large divergence indicates increasingly confident errors on validation slides. The saved early checkpoint protects the reported test result from using the visibly overfit last epoch.

Class Specific Errors

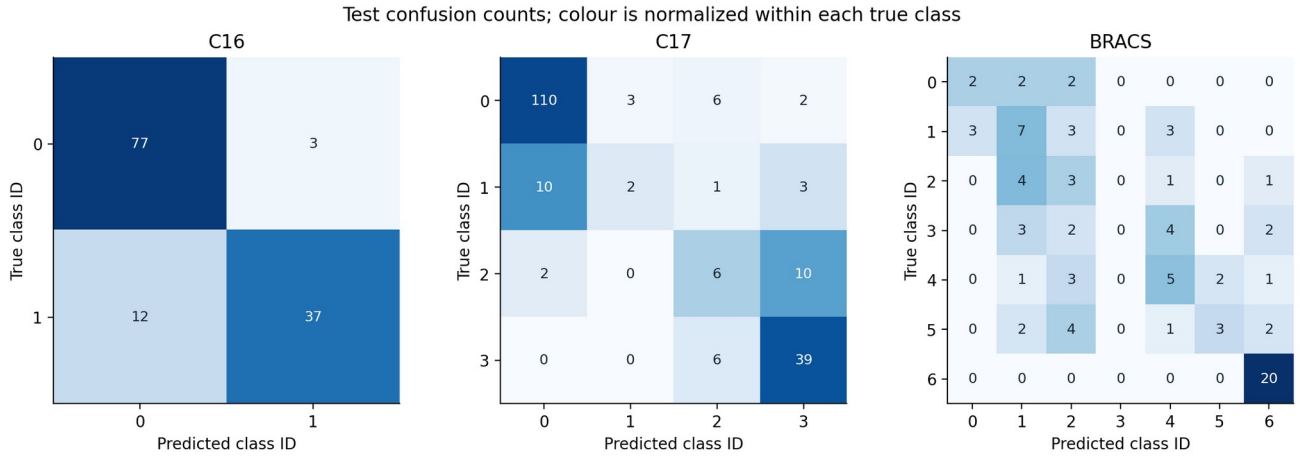


Figure 12. Held-out confusion matrices. Rows are true classes and columns predicted classes. Numbers are slide counts; colour is normalized within each row so minority-class errors remain visible. Dark diagonal cells indicate high recall.

C16 Error Structure

C16 classifies 114 of 129 test slides correctly. Of 80 class-0 slides, 77 are classified as class 0 and 3 as class 1. Of 49 class-1 slides, 37 are classified as class 1 and 12 as class 0. The main observed error direction is therefore missed class-1 cases. Class-1 precision is 92.50%, while recall is 75.51%: predictions of class 1 are usually correct, but nearly one quarter of the true class-1 slides are missed. This tradeoff cannot be inferred from the 88.37% overall accuracy alone.

Table 9. C16 per-class held-out metrics

ID	Support	Precision	Recall	F1	AUROC
0	80	0.8652	0.9625	0.9112	0.9224
1	49	0.9250	0.7551	0.8315	0.9224

The binary one-versus-rest AUCs are equal because each of the two class probabilities complements the other. The AUC of 0.9224 indicates useful ranking, while the argmax decisions still make 15 errors. No alternative decision threshold has been tuned on the test data.

C17 Error Structure

Table 10. C17 per-class held-out metrics

ID	Support	Precision	Recall	F1	AUROC
0	121	0.9016	0.9091	0.9053	0.9113
1	16	0.4000	0.1250	0.1905	0.6111
2	18	0.3158	0.3333	0.3243	0.8056
3	45	0.7222	0.8667	0.7879	0.9626

C17 classifies 157 of 200 slides correctly. Class 0 contributes 121 slides, more than 60% of the test set, and 110 are correctly recognized. Class 3 contributes another 45 slides with 39 correct. Those two groups account for 149 of the 157 correct predictions. In contrast, class 1 has only 2 correct predictions among 16 slides, and class 2 has 6 among 18. Overall accuracy therefore gives a substantially more favorable impression than the macro F1 of 0.5520.

Ten class-1 slides are predicted as class 0, three as class 3 and one as class 2. Class 2 is most often confused with class 3: 10 of 18 examples move in that direction. The model predicts class 1 only five times, correctly twice. Its low recall is accompanied by a one-versus-rest AUROC of only 0.6111. Class 2 has a higher AUC of 0.8056 but only 33.33% recall under argmax. The contrast illustrates that a useful ranking score for one class does not guarantee that it wins the multiclass decision.

The 27 shared training and validation patients can make validation more optimistic than a patient-independent partition. Test patients and centres are separate under the implemented split. The decrease from validation macro F1 of 0.7856 to test macro F1 of 0.5520 is therefore interpreted alongside this dependence and a validation support of only two class-1 slides. The present results do not isolate the effects of patient overlap, centre shift and sampling variation.

BRACS Error Structure

Table 11. BRACS per-class held-out metrics

ID	Support	Precision	Recall	F1	AUROC
0	6	0.4000	0.3333	0.3636	0.8229
1	16	0.3684	0.4375	0.4000	0.7580
2	9	0.1765	0.3333	0.2308	0.6681
3	11	0.0000	0.0000	0.0000	0.6073
4	12	0.3571	0.4167	0.3846	0.8029
5	12	0.6000	0.2500	0.3529	0.8423
6	20	0.7692	1.0000	0.8696	0.9970

BRACS correctly classifies 40 of 86 slides. Class 6 is recognized in all 20 test examples, accounting for half of the model's correct decisions. However, its precision is 0.7692 because six slides from other classes are also predicted as class 6. Perfect recall for this class in a finite test sample is not evidence of universally perfect performance.

Class 3 has no correct predictions among 11 slides, and the classifier never selects class 3. Its true slides are assigned to classes 1, 2, 4 and 6. Its F1 is therefore zero despite a one-versus-rest AUROC of 0.6073. Apart from class 6, all nonzero recalls lie between 25.00% and 43.75%. The macro F1 of 0.3716 summarizes this broad weakness without identifying which architectural or data-related factor caused it.

Ranking and Decision Thresholds

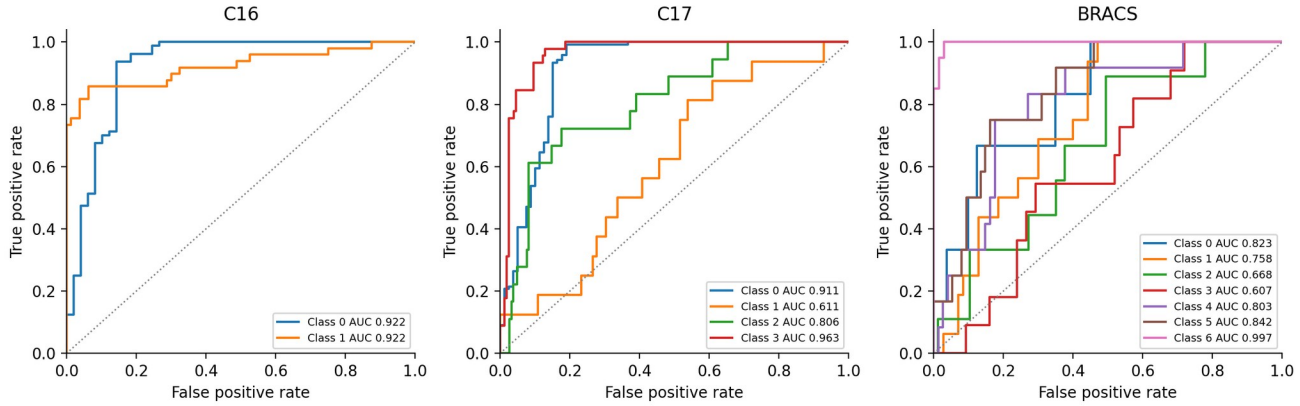


Figure 13. One-versus-rest receiver operating characteristic (ROC) curves recomputed from the saved test probabilities. Each curve treats one class as positive and all other classes as negative. The dotted diagonal indicates chance ranking. Curve labels show class-specific AUROC rather than its macro average.

ROC curves compare true-positive and false-positive rates across thresholds. For C17, the class-1 curve is much weaker than class 3. For BRACS, class 6 is almost perfectly ranked, while classes 2 and 3 are substantially weaker. This pattern is consistent with the per-class table, but the thresholds traced by a one-versus-rest curve are not a replacement for the fixed multiclass argmax rule used to report accuracy and F1.

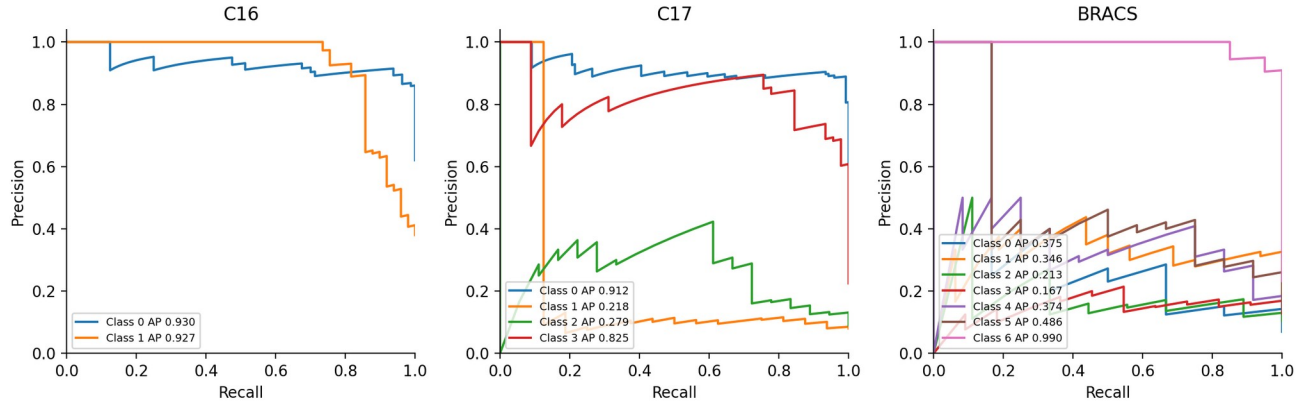


Figure 14. One-versus-rest precision–recall curves from saved probabilities. Average precision (AP) summarizes each class curve and was not used to select checkpoints. Different class prevalences imply different chance precision levels.

The precision–recall curves and average precision (AP) values are descriptive summaries of the saved test probabilities. They were not used to fit thresholds or choose models. Class prevalence differs across datasets, so the curves do not represent a single common operating environment or establish clinical utility.

Qualitative Attention on Whole Slide Images

Two C16 examples use the same epoch-three checkpoint as the baseline table, verified by its recorded SHA-256 fingerprint. The negative slide test_067 contains 11,926 patches and receives tumour probability 0.0449. The

positive slide test_068 contains 15,037 patches and receives probability 0.9877. Tumour contours are read from the supplied Extensible Markup Language (XML) annotations. These examples were selected after baseline evaluation for qualitative illustration and are not an unbiased localization benchmark.

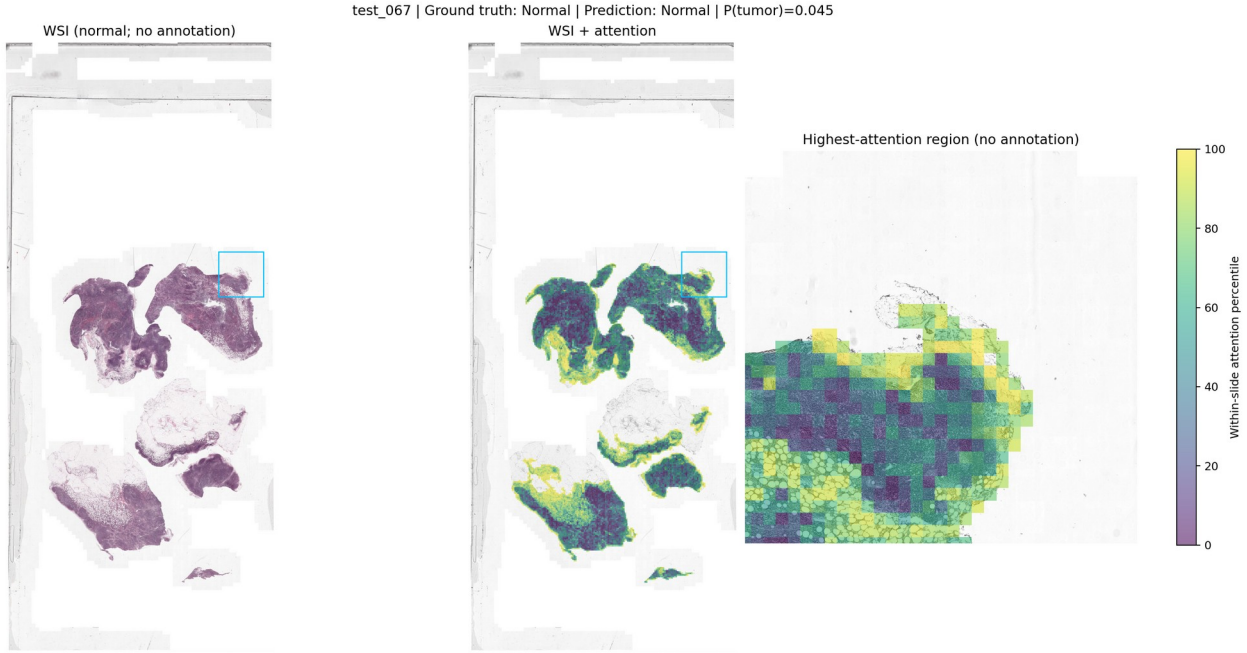


Figure 15. C16 negative slide test_067. The original tissue view and attention overlay show the relative distribution of pooling weights in a correctly classified negative slide. The color scale is a within-slide percentile, so brightly colored tissue need not represent disease.

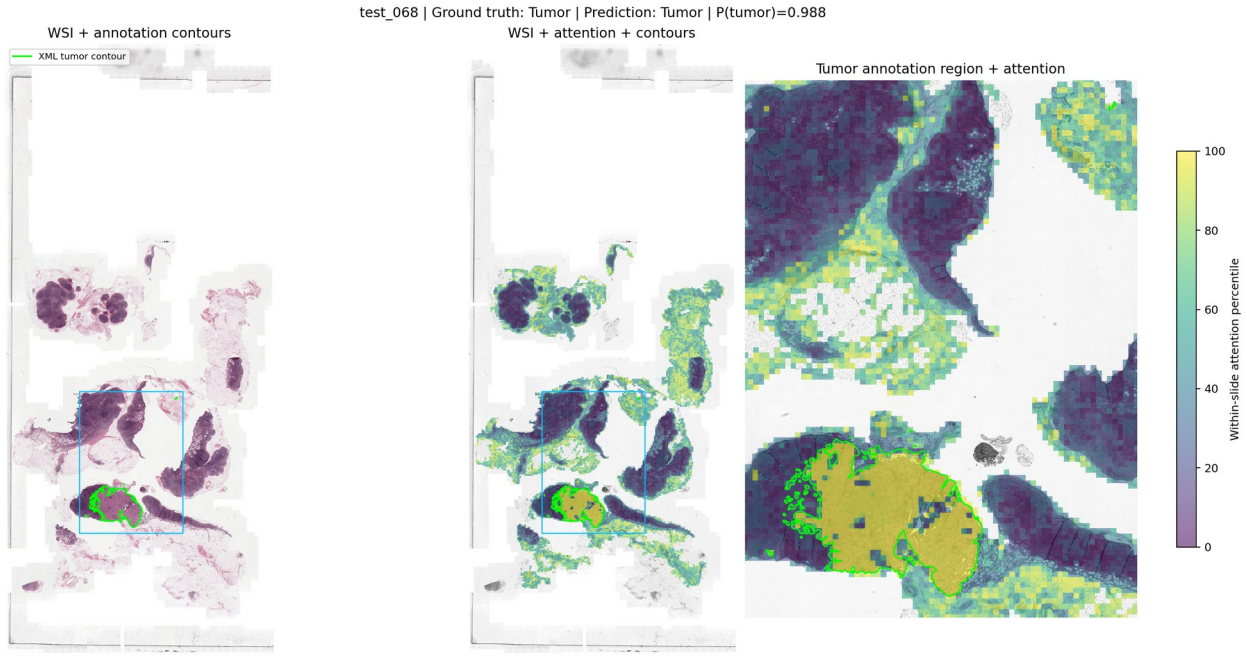


Figure 16. C16 positive slide test_068. Green curves mark the supplied tumour annotation contours, the blue box indicates the enlarged region, and the overlay shows within-slide attention percentiles. Some high-weight regions coincide visually with annotated tissue, but this selected example does not establish localization sensitivity or specificity.

The rendering uses level-zero coordinates as patch top-left corners and an assumed 512-pixel patch footprint. These conventions support an approximate spatial display and are not independently verified extraction metadata. No quantitative overlap score, lesion-detection result or segmentation claim is derived. Because ranks are normalized separately within each slide, the colors cannot be compared as absolute disease probabilities across images.

6 Research Proposals

Sensitivity Controlled Attention and Mean Pooling

The first proposal asks whether sensitivity to suppressing high-attention evidence can inform the amount of distributed context retained in a slide representation. If a prediction depends strongly on a small subset of instances, a mean contribution could reduce that dependence. The implemented rule derives its mixing coefficient from a label-free intervention and retains the ABMIL parameter count. Its hypothesis concerns the usefulness of this control signal, rather than the novelty of pooling mixtures themselves.

Attention manipulation is established in prior work. ACMIL uses multiple branches and stochastic masking [7], while MHIM-MIL uses teacher-guided masking [6]. Counterfactual attention regularization for multiple instance learning (CAR-MIL) learns an auxiliary attention branch under proximity and prediction-difference objectives [13]. In contrast, the present prototype uses one scorer and a deterministic suppression probe to control mean pooling, with no auxiliary training loss. This distinction defines the experiment without establishing global novelty. The method was selected after baseline test results were available; extension development used validation data.

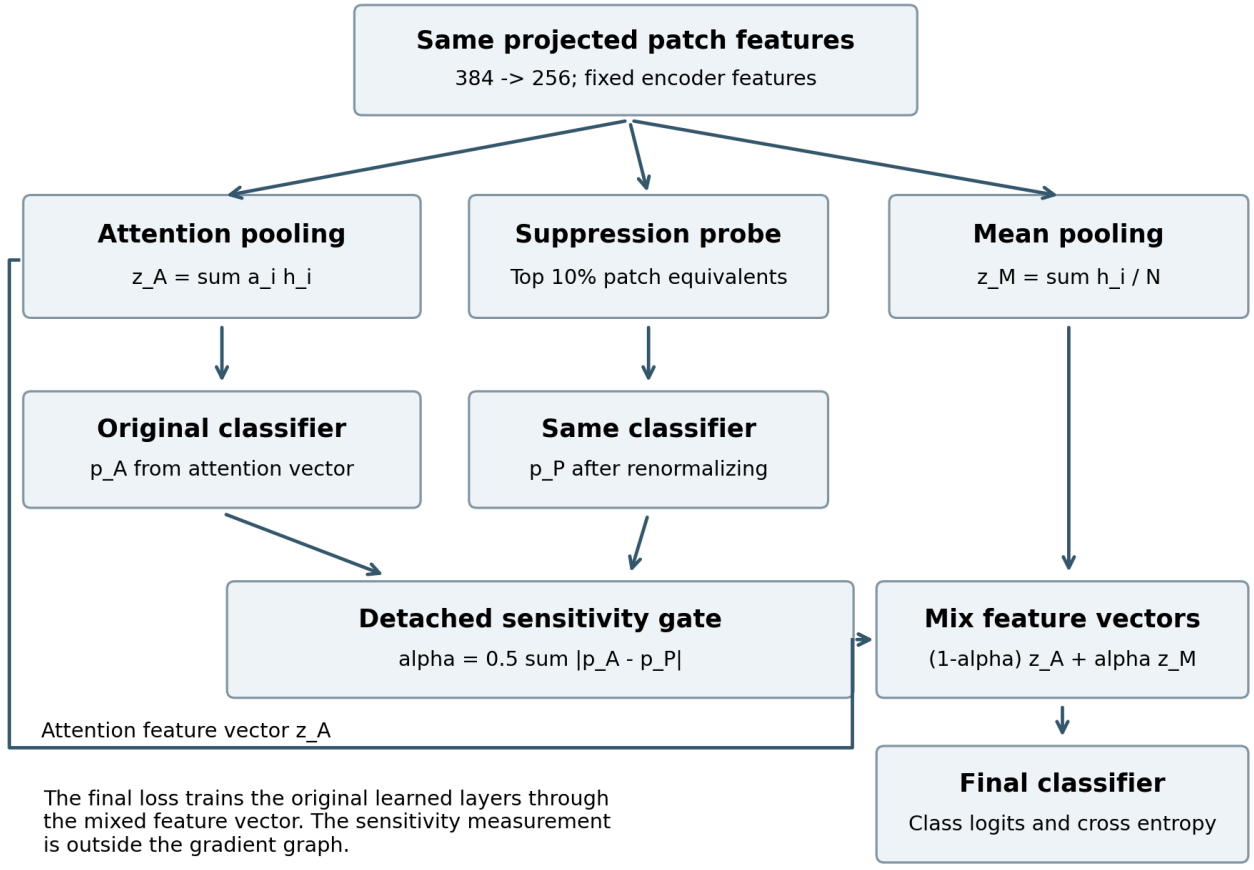


Figure 17. The implemented extension. A shared projection and attention scorer supply the attention vector and a suppression probe. The change between two class distributions controls the final feature mixture. The classifier weights are shared across all three classifier evaluations.

Suppression Probe and Adaptive Mixture

For each valid patch, the shared projection and attention scorer produce h_i and s_i. Patch softmax gives a_i. Aggregation produces the attention-weighted vector z_A and valid-patch mean z_M. The shared classifier g maps z_A to the attention-only class distribution p_A.

$$z_A = \sum_i a_i h_i, z_M = \frac{1}{N}\sum_i h_i, p_A = \mathit{softmax}(g(z_A))$$

The probe suppresses k = min(N - 1, ceil(0.10 N)) highest-score patch equivalents for N greater than one. Suppression is based on patch count, not a fixed amount of attention mass. If patches tie at the cutoff, the remaining suppression is shared equally among the tied patches. The resulting boundary operation can be fractional rather than literal patch removal. It preserves symmetry under patch reordering and ensures at least some evidence remains. Single-patch bags are not perturbed.

$$\tilde{a}_i = \mathit{softmax}_i(s + \log r), z_P = \sum_i \tilde{a}_i h_i, p_P = \mathit{softmax}(g(z_P))$$

Here r_i is the retained fraction of a patch. Fully removed and padded positions receive zero weight. Calculating the probed distribution from the original scores avoids dividing tiny surviving softmax probabilities. The diagnostic computation runs without gradient tracking. Its result is a total-variation distance between the attention-only and probed class distributions.

$$\alpha = \frac{1}{2}\sum_c |p_{A,c} - p_{P,c}|, \quad 0 \le \alpha \le 1$$

The final slide vector mixes attention pooling with the valid-patch mean. Larger probability changes increase the mean contribution. Ordinary cross-entropy trains the shared layers through the mixed vector. The coefficient is recomputed during training and inference. It is detached from gradient computation, so there is no direct derivative through the diagnostic, although parameter updates influence subsequent diagnostic values.

$$z = (1 - \alpha) z_A + \alpha z_M, \quad \ell = g(z)$$

The mixture operates on features. It is not a weighted average of class probabilities. Because the classifier is linear with a shared bias, mixed features imply mixed logits; class softmax is then applied to those logits. This distinction matters when explaining individual probability changes.

Effective Patch Weights and Interpretation

$$w_i = (1 - \alpha) a_i + \frac{\alpha}{N}, \quad z = \sum_i w_i h_i$$

The mean branch uses the same projected features. It adds no independent image information; it smooths attention weights toward a uniform distribution. At alpha zero, the model reduces to attention pooling. At alpha one, it uses mean pooling. Returned raw attention describes the attention branch; the full hybrid pooling contribution is w_i. Alpha describes sensitivity to this particular probe and is not calibrated uncertainty, a lesion probability or a causal effect.

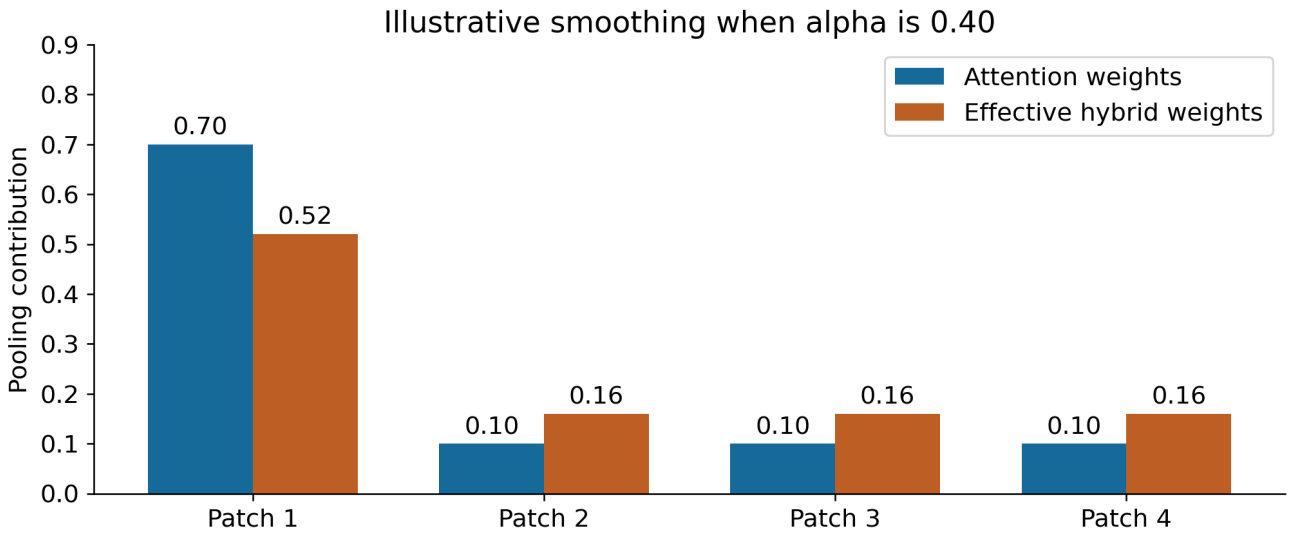


Figure 18. An illustrative four-patch example, not measured pathology data. With attention weights 0.70, 0.10, 0.10, 0.10 and alpha 0.40, effective weights become 0.52, 0.16, 0.16, 0.16. The diagnostic gate therefore changes how strongly the most attended patch dominates pooling.

Implementation Compared With the Baseline

Table 12. Architectural comparison

Property	ABMIL	Fixed mixture	Fragility guided
Patch representation	384 to 256 projection	Same	Same
Attention scorer	Gated hidden dimension 128	Same	Same
Pooling	Attention weighted sum	0.5 attention plus 0.5 mean	Slide-specific attention and mean
Mixing weight	No mean branch	Fixed 0.5	Detached probability sensitivity
Additional trainable parameters	0 reference	0	0
Classifier evaluations per forward	1	1	3 shared-weight evaluations
Feature projection per forward	Once	Once	Once
Loss	Cross-entropy	Cross-entropy	Cross-entropy; no consistency penalty
Extra evidence probe	None	None	Top 10% patch equivalents
Real-data evidence	30 epochs and held-out test	30 epochs and held-out test	30 epochs and held-out test

The extension reuses the layers inherited from ABMIL in model_fragility.py. suppress_top_fraction performs symmetric cutoff handling. FragilityGuidedMIL.forward computes original and probed distributions, detaches alpha and classifies the mixed vector. research/train_extension.py preserves the original training recipe, archives

sources and checks exact split identity. The original model.py, train.py, model_hybrid.py, engine.py, dataloader.py and utils.py remain unchanged.

Computational Cost

For N patches, input dimension D = 384, projection dimension d = 256 and attention dimension h = 128, the dominant baseline dense operations scale as O(NDd + Ndh + dC). Stored activations scale linearly with N for fixed dimensions, including O(Nd) projected features. The extension reuses the projection and scorer, adds reductions over patches, a top-k selection and two shared-classifier evaluations. The selection cost depends on the library implementation; no measured speedup, latency ratio or memory benchmark is asserted. Both architectures avoid the dense N-by-N attention matrix of a standard full self-attention operator.

Sparse Evidence and Failure Modes

A large change after removing salient patches can mean the model relied on a shortcut, but it can also mean the removed patches contained the only genuine disease evidence. Uniform context can then dilute a correct focal signal. Conversely, a confidently wrong model can have a small sensitivity score. Softmax saturation, repeated evidence and the chosen suppression budget can also affect alpha. These mechanisms prevent interpreting a high alpha as a reliable instruction to distrust attention.

Normalized Attention Entropy Regularization

The second proposal regularizes attention dispersion directly. The motivation is that a slide-level classification objective may concentrate weight on very few instances even when diagnostic information is distributed. Let $H(a) = -\Sigma_i a_i \log a_i$. For $N > 1$, dividing by $\log N$ gives a normalized entropy in [0, 1]; the term is defined as zero for a single-instance bag. Minimizing cross-entropy minus a nonnegative coefficient times this entropy rewards broader attention without changing the pooling architecture or adding trainable parameters.

A nonnegative coefficient β controls the strength of the entropy term. Its negative sign in the minimized objective rewards higher entropy. At $\beta = 0$ the objective reduces to the ABMIL baseline. Normalization makes a common coefficient across bag sizes more interpretable; it does not imply identical effects across tissue distributions.

This proposal intervenes on the training objective, whereas sensitivity-controlled pooling changes the representation during both training and inference. Unlike masking and branch-diversity objectives [6, 7], it directly constrains the concentration statistic. It also introduces a clear failure mode: forcing weight onto non-diagnostic tissue can dilute a focal lesion. The proposal is unimplemented and has no measured performance result; no regularization coefficient has been selected.

$$L = CE - \beta \frac{H(a)}{\log N}, \qquad \beta \geq 0, \qquad N > 1$$

7 Extension Evaluation

Full Training and Held Out Protocol

The full comparison retains 30 epochs, seed 42, batch size one, the original feature files and exact slide partitions, AdamW settings and validation selection rule. The probe fraction remains 0.10. Three adaptive-pooling runs and three pure-mean controls were completed before any of their held-out predictions were generated. Each selected checkpoint was then evaluated once with the guarded research evaluator. No model setting was selected from these test outcomes.

ABMIL results reuse the group baseline checkpoints in Section 5. The fixed 50/50 controls reuse the completed October 28 Mac experiments, with matching split identifiers and checkpoint fingerprints. The new adaptive and mean runs also use the Mac CPU environment. Thus the primary baseline comparison holds the training recipe and data constant but crosses platforms. An earlier Mac C17 ABMIL run is retained as an additional environment reference; it is not substituted for the group baseline after observing extension results. Fixed-mixture records retain aggregate test metrics but not individual probabilities, so class-specific comparisons below concern ABMIL and adaptive pooling.

Held Out Classification and Pooling Controls

Table 13. Held-out comparison after 30 training epochs. One seed per configuration.

Dataset	Pooling	Epoch	Accuracy %	Macro AUROC	Macro F1	CE loss
C16	ABMIL	3	88.37	0.9224	0.8714	0.2700
C16	Fixed 50/50	3	90.70	0.9128	0.8986	0.2738
C16	Mean	16	65.89	0.6107	0.5831	0.6551
C16	Fragility	12	86.05	0.9069	0.8480	0.3799
C17	ABMIL	21	78.50	0.8226	0.5520	0.7969
C17	Fixed 50/50	7	81.00	0.8640	0.4922	0.5054
C17	Mean	2	60.50	0.7045	0.1885	0.9928
C17	Fragility	19	78.50	0.8428	0.5218	0.6753
BRACS	ABMIL	3	46.51	0.7855	0.3716	1.4110
BRACS	Fixed 50/50	12	44.19	0.7860	0.3671	1.4786
BRACS	Mean	20	36.05	0.6529	0.2648	1.8416
BRACS	Fragility	19	33.72	0.7051	0.2316	2.1113

On C16, adaptive pooling achieves accuracy 86.05%, macro AUROC 0.9069, macro F1 0.8480 and cross-entropy 0.3799. Relative to the primary ABMIL result, the changes are -2.33 percentage points in accuracy and -0.0234 in macro F1. The fixed-mixture and pure-mean controls achieve macro F1 0.8986 and 0.5831, respectively. These quantities describe the selected single-run checkpoints, rather than an average over initializations.

On C17, adaptive pooling achieves accuracy 78.50%, macro AUROC 0.8428, macro F1 0.5218 and cross-entropy 0.6753. Relative to the primary ABMIL result, the changes are +0.00 percentage points in accuracy and -0.0302 in macro F1. The fixed-mixture and pure-mean controls achieve macro F1 0.4922 and 0.1885, respectively. These quantities describe the selected single-run checkpoints, rather than an average over initializations.

On BRACS, adaptive pooling achieves accuracy 33.72%, macro AUROC 0.7051, macro F1 0.2316 and cross-entropy 2.1113. Relative to the primary ABMIL result, the changes are -12.79 percentage points in accuracy and -0.1400 in macro F1. The fixed-mixture and pure-mean controls achieve macro F1 0.3671 and 0.2648, respectively. These quantities describe the selected single-run checkpoints, rather than an average over initializations.

The adaptive rule does not improve macro F1 on any dataset. The fixed mixture has the highest C16 macro F1, whereas ABMIL has the highest macro F1 on C17 and BRACS among the tested operators. Pure mean performs substantially below ABMIL on every dataset. Thus retaining some attention is useful in this comparison, but the proposed sensitivity rule does not allocate the mean contribution more effectively than the reference methods. On C17, the fixed mixture improves accuracy and ranking while reducing macro F1, demonstrating that the preferred operator depends on the metric.

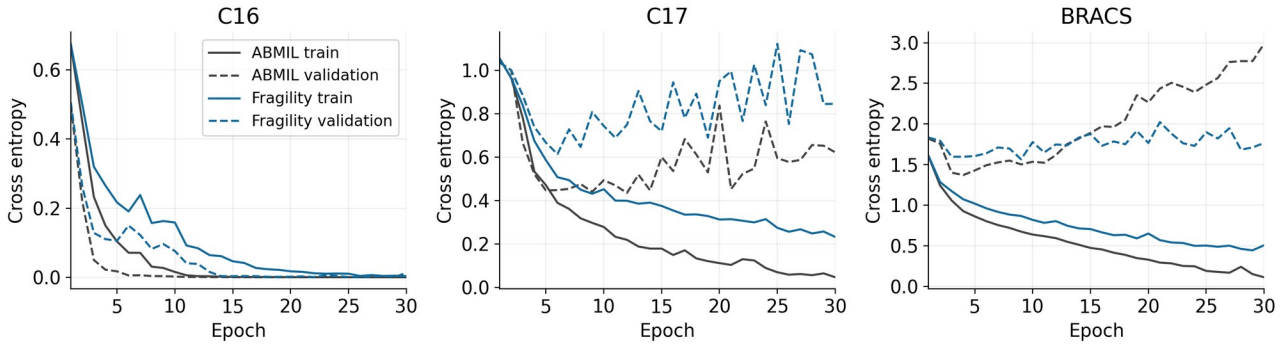


Figure 19. Full 30-epoch ABMIL and adaptive-pooling histories. Color identifies the method; solid lines show training cross-entropy and dashed lines validation cross-entropy. Each dataset has its own vertical scale. The curves describe optimization and validation behavior; test data do not select epochs.

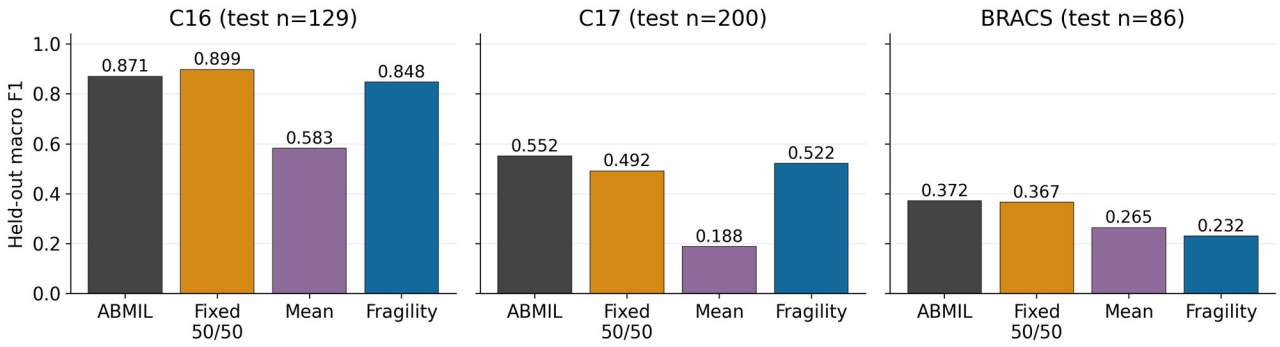


Figure 20. Held-out macro F1 for ABMIL, fixed 50/50 attention–mean pooling, pure mean and adaptive pooling. All panels use the same zero-to-one scale. Labels are single-seed point estimates; no error bars or significance claim are implied. Test sample sizes are shown in each panel.

Class Specific Effects

Table 14. Class-specific held-out comparison. Recall and F1 are fractions; support counts slides.

Dataset	ID	Support	ABMIL recall	Adaptive recall	ABMIL F1	Adaptive F1
C16	0	80	0.9625	0.9250	0.9112	0.8916
C16	1	49	0.7551	0.7551	0.8315	0.8043
C17	0	121	0.9091	0.8926	0.9053	0.8963
C17	1	16	0.1250	0.0000	0.1905	0.0000
C17	2	18	0.3333	0.3333	0.3243	0.3871

Dataset	ID	Support	ABMIL recall	Adaptive recall	ABMIL F1	Adaptive F1
C17	3	45	0.8667	0.9556	0.7879	0.8037
BRACS	0	6	0.3333	0.1667	0.3636	0.1538
BRACS	1	16	0.4375	0.5625	0.4000	0.4091
BRACS	2	9	0.3333	0.2222	0.2308	0.1905
BRACS	3	11	0.0000	0.0000	0.0000	0.0000
BRACS	4	12	0.4167	0.0000	0.3846	0.0000
BRACS	5	12	0.2500	0.1667	0.3529	0.1538
BRACS	6	20	1.0000	0.7500	0.8696	0.7143

The same test support is used within every class comparison. An increase in overall accuracy can coexist with lower macro F1 when decisions shift toward frequent classes. The confusion matrices and equal-weight class average therefore provide necessary context for the aggregate accuracy differences. No class threshold is tuned on these test predictions.

On C16, positive-class recall remains 37/49, while correctly recognized negative slides decrease from 77/80 to 74/80. On C17, the unchanged 78.50% accuracy hides a redistribution of errors: class-1 recall falls from 2/16 to 0/16, while class-3 recall rises from 39/45 to 43/45. On BRACS, adaptive pooling fails to recognize class 4 at all (0/12 versus 5/12 for ABMIL), and class-6 recall declines from 20/20 to 15/20. The small class-1 recall gain does not compensate for the broader deterioration in macro F1.

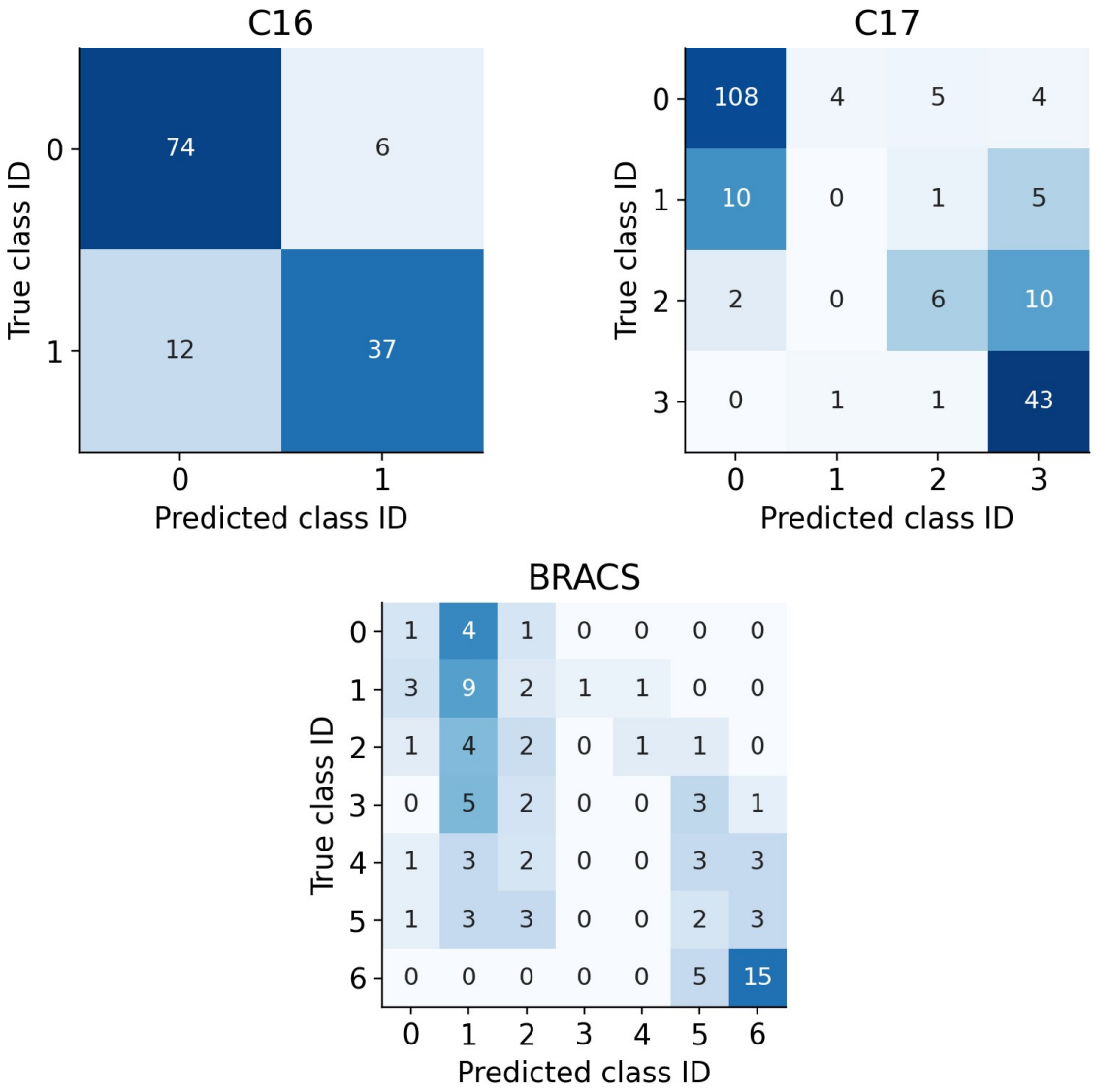


Figure 21. Adaptive-pooling held-out confusion matrices. Rows are true class IDs and columns are predictions. Cell labels are slide counts; shading represents within-row recall on a common zero-to-one scale. The ABMIL matrices in Figure 12 provide the corresponding baseline reference.

Adaptive Weight Diagnostics

Table 15. Adaptive weights on the held-out slides. Suppressed mass refers to original attention, not patch count.

Dataset	Test slides	Mean alpha	Median alpha	Minimum	Maximum	Mean suppressed mass
C16	129	0.4445	0.4319	0.0000	0.9981	0.9652
C17	200	0.3761	0.3477	0.0000	0.9999	0.9226
BRACS	86	0.5142	0.6203	0.0004	0.9831	0.9371

Table 16. Gate association with correctness and within-checkpoint prediction changes.

Dataset	Incorrect slides	Mean alpha incorrect	Correct slides	Mean alpha correct	Attention to mixture flips
C16	18	0.6266	111	0.4150	9
C17	43	0.5684	157	0.3235	14
BRACS	57	0.6087	29	0.3285	26

The diagnostic varies across test slides, confirming that the implemented rule uses slide-specific mixtures. The removed fraction is fixed by patch count, whereas removed attention mass depends on concentration. Alpha is the total-variation change in the classifier probabilities under this particular probe. Correctness groups are descriptive summaries formed after prediction, not inputs to the gate or evidence of calibrated uncertainty. Label flips compare the attention-only and final mixed branches within the same trained adaptive model, rather than two separately trained models.

The mean suppressed attention masses are 96.52% on C16, 92.26% on C17 and 93.71% on BRACS, despite a suppression budget of approximately 10% of patches. This concentration makes the probe a substantial intervention on the pooled evidence. Mean alpha is higher on incorrect than correct predictions in all three cohorts, but that association does not validate the prescribed response of adding uniform context. The two quantities concern sensitivity and decision quality, respectively, and the measured classification results show that their relationship is insufficient for this rule.

Within the trained adaptive checkpoints, applying the final mixture introduces nine errors and repairs none on C16, introduces eight and repairs four on C17, and introduces thirteen and repairs two on BRACS relative to each checkpoint's attention-only branch. The remaining C17 and BRACS flips switch between incorrect classes. These are paired branch diagnostics with a shared representation trained for mixed prediction, not independent ABMIL controls. The prescribed mixture reduces accuracy relative to each checkpoint's attention-only readout; this comparison concerns decision behavior within the trained models.

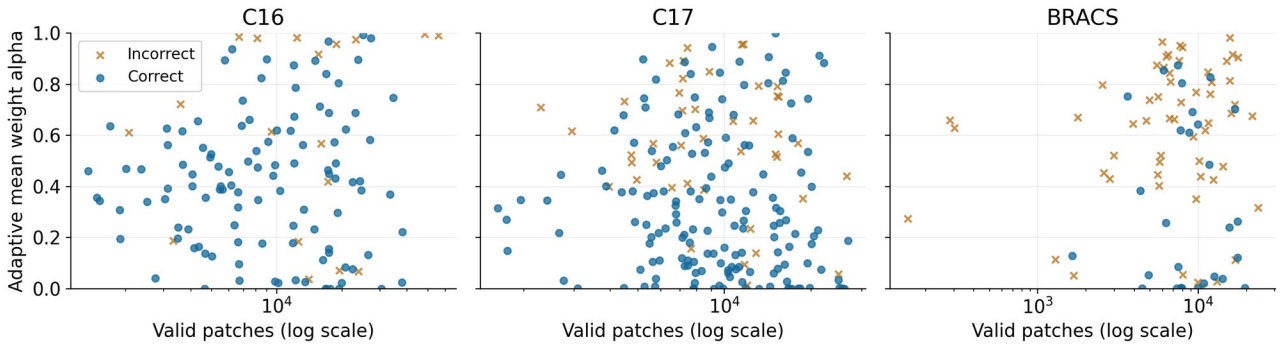


Figure 22. Adaptive mean weight versus valid patch count on held-out slides. Circles identify correct predictions and crosses errors; the horizontal axis is logarithmic and the vertical scale is shared. These plots are descriptive and were not used to choose a diagnostic threshold or probe setting.

8 Discussion

Baseline performance varies substantially across the three label spaces. C16 supports strong discrimination but still misses 12 of 49 positive slides at the fixed decision rule. C17 accuracy is dominated by frequent classes, whereas its macro F1 and minority-class recall reveal a different error profile. BRACS combines heterogeneous class errors with a widening training–validation loss gap. These observations motivate investigation of aggregation but do not identify attention concentration as the cause of the failures.

Focused Evidence and Distributed Context

Fragility-guided pooling changes the allocation of weight within the same learned feature space. It cannot restore information missing from the fixed encoder. Fixed 50/50 and pure-mean controls expose the difference between adaptive allocation and uniform smoothing. The comparison does not support attributing every change relative to ABMIL to the sensitivity signal alone: the selected checkpoints differ, and the primary baselines cross training environments. Uniform context can preserve distributed morphology while also introducing irrelevant tissue or acquisition effects.

Class Imbalance and Feature Limitations

The C17 class-1 recall of 12.50% and class-2 recall of 33.33% show that aggregate accuracy is insufficient to characterize recognition across classes. Loss weighting, visual representation and aggregation could each alter these outcomes. None is isolated by the present experiments. Numerical label order alone also provides no justification for an ordinal loss or a clinical interpretation of adjacent identifiers.

Statistical Scope

Each reported configuration uses seed 42, leaving across-initialization variability unquantified. Patient dependence further limits interpretation of slide-level sample sizes, especially in C17 validation. No confidence interval or significance claim is supported by these single-run point estimates. The implemented slide-level partitions also differ from the full original challenge evaluation procedures, and no patient-level staging result is reported. The earlier Mac C17 ABMIL baseline selected epoch 19 and achieved accuracy 73.00%, macro AUROC 0.8325 and macro F1 0.5426, compared with the primary Windows epoch-21 result of 78.50%, 0.8226 and 0.5520. This retained environment comparison indicates that a common integer seed does not guarantee identical trajectories across platforms.

Sensitivity and Uncertainty

Suppressing a fixed fraction of patches removes a variable amount of attention mass. Total variation also depends on the probability scale, so confidence can change alpha without changing the predicted class. The full held-out comparison measures classification performance under the clean feature protocol. No common-intervention perturbation benchmark or lesion-localization evaluation is performed. The results therefore do not establish robustness, causal explanation faithfulness or calibrated uncertainty.

9 Reproducibility and Research Artifacts

Read-only input audits verify feature dimensions, class coverage, disjoint slide identifiers, finite values and recorded fingerprints. Baseline software checks exercise mask handling, permutation invariance and the complete synthetic training pipeline. The archived extension log records seven tests covering the zero-probe ABMIL endpoint, parameter equality, padding, ties, single-instance bags, numerical stability, normalized effective weights, finite gradients and synthetic workflows. The evaluation tests additionally cover source snapshots, debugging-run rejection and duplicate-evaluation protection. These checks establish software properties rather than pathology performance.

The accompanying source and results archive contains training and evaluation programs, configurations, notebooks, selected checkpoints, saved predictions and figures. Every full run retains configuration, environment, partitions, history, summary and checkpoint records. Research runs also archive their source snapshot. Primary baselines and development pilots used Python 3.9.5 and PyTorch 2.8.0+cpu on the recorded Intel Core i7-1065G7 Windows CPU; full adaptive and mean runs used Python 3.12.14 and PyTorch 2.8.0 on the Mac CPU, with two threads per process and no accelerator. Fixed-mixture controls used the same Mac Python/PyTorch versions. Original baseline test inference used one thread; the new evaluator used two. Cross-platform numerical and random-number behavior limits exact comparability despite matching seeds and settings.

Measured single-run training wall times, including runner startup, were C16 fragility: 140.1 s; C17 fragility: 168.7 s; BRACS fragility: 228.7 s; C16 mean: 123.8 s; C17 mean: 150.3 s; BRACS mean: 230.3 s. These are individual

30-epoch runs, not means over repetitions. Historical baseline and fixed-mixture wall times, peak memory and energy consumption were not recorded; no speed ratio is inferred.

The project repository is https://github.com/AaronEmmanuel-UofT/ECE1512-Project-A. The baseline and pilot source archive was audited at commit 818f347e437c5dc7c60eaad56ea75a74d804758a. The full comparison is recorded under results/full_comparison_20261102, with a frozen protocol, source snapshots and an artifact manifest. Large feature files are obtained separately from [12].

Feature Provenance and Experimental Chronology

Feature artifacts originate from LeonYe/ECE1512-WSI-Feature [12], revision 6df7ef4b42b7ecd11633722a392a1efc6d80bcc7. Provenance records retain file sizes and complete SHA-256 fingerprints. The supplied data-loading and metric utilities are preserved with attribution. An earlier two-epoch debugging notebook exposed test results before its guard was corrected; those outputs are excluded from the final baseline tables. The archived full-run baselines were evaluated after validation-based checkpoint selection. Subsequent extension development occurred with knowledge of baseline test outcomes, which limits a strictly prospective interpretation of the research hypotheses.

10 Conclusions

ABMIL provides a slide-classification reference on the three feature datasets, with held-out accuracy 88.37% on C16, 78.50% on C17 and 46.51% on BRACS. Sensitivity-controlled pooling preserves the trainable parameter count while changing the balance between focused and uniform evidence. Its full held-out results and the fixed-mixture and pure-mean controls show lower macro F1 than ABMIL for the adaptive rule on all three datasets. The largest decline occurs on BRACS, where macro F1 falls from 0.3716 to 0.2316. A fixed 50/50 mixture is preferable on C16 in this single-seed comparison, and ABMIL retains the highest macro F1 on C17 and BRACS. Suppression sensitivity alone does not justify the proposed allocation of uniform context. Normalized attention entropy regularization remains a distinct, unimplemented objective-level proposal. The measured single-seed comparisons do not establish statistical superiority, biological explanation faithfulness or clinical validity. The principal finding is a task-dependent account of how a defined sensitivity rule changes classification and pooling behavior under preserved features and partitions.

References

[1] M. Ilse, J. M. Tomczak, and M. Welling, “Attention-based Deep Multiple Instance Learning,” in Proceedings of the International Conference on Machine Learning, Proceedings of Machine Learning Research, vol. 80, pp. 2127–2136, 2018. Full text.

[2] M. Y. Lu, D. F. K. Williamson, T. Y. Chen, R. J. Chen, M. Barbieri, and F. Mahmood, “Data-efficient and weakly supervised computational pathology on whole-slide images,” Nature Biomedical Engineering, vol. 5, pp. 555–570, 2021. doi: 10.1038/s41551-020-00682-w.

[3] B. Li, Y. Li, and K. W. Eliceiri, “Dual-Stream Multiple Instance Learning Network for Whole Slide Image Classification With Self-Supervised Contrastive Learning,” in Proceedings of the Conference on Computer Vision and Pattern Recognition, pp. 14318–14328, 2021. Full text.

[4] Z. Shao, H. Bian, Y. Chen, Y. Wang, J. Zhang, X. Ji, and Y. Zhang, “TransMIL: Transformer based Correlated Multiple Instance Learning for Whole Slide Image Classification,” in Advances in Neural Information Processing Systems, vol. 34, pp. 2136–2147, 2021. Full text.

[5] H. Zhang et al., “DTFD-MIL: Double-Tier Feature Distillation Multiple Instance Learning for Histopathology Whole Slide Image Classification,” in Proceedings of the Conference on Computer Vision and Pattern Recognition, pp. 18802–18812, 2022. Full text.

[6] W. Tang, S. Huang, X. Zhang, F. Zhou, Y. Zhang, and B. Liu, “Multiple Instance Learning Framework with Masked Hard Instance Mining for Whole Slide Image Classification,” in Proceedings of the International Conference on Computer Vision, pp. 4078–4087, 2023. Full text.

[7] Y. Zhang, H. Li, Y. Sun, S. Zheng, C. Zhu, and L. Yang, “Attention-Challenging Multiple Instance Learning for Whole Slide Image Classification,” in Computer Vision – European Conference on Computer Vision 2024, Lecture Notes in Computer Science, vol. 15111, pp. 125–143, Springer, 2025. doi: 10.1007/978-3-031-73668-1_8.

[8] R. J. Chen et al., “Towards a general-purpose foundation model for computational pathology,” Nature Medicine, vol. 30, pp. 850–862, 2024. doi: 10.1038/s41591-024-02857-3.

[9] G. Litjens et al., “1399 H&E-stained sentinel lymph node sections of breast cancer patients: the CAMELYON dataset,” GigaScience, vol. 7, no. 6, article giy065, 2018. doi: 10.1093/gigascience/giy065. Source

[10] P. Bándi et al., “From Detection of Individual Metastases to Classification of Lymph Node Status at the Patient Level: The CAMELYON17 Challenge,” IEEE Transactions on Medical Imaging, vol. 38, no. 2, pp. 550–560, 2019. doi: 10.1109/TMI.2018.2867350. Source

[11] N. Brancati et al., “BRACS: A Dataset for BReAst Carcinoma Subtyping in H&E Histology Images,” Database, vol. 2022, article baac093, 2022. doi: 10.1093/database/baac093. Source

[12] LeonYe, “ECE1512 WSI Feature,” Hugging Face dataset repository, revision 6df7ef4b42b7ecd11633722a392a1efc6d80bcc7, accessed Oct. 27, 2026. Source

[13] I. Chraki, P. Marza, S. Christodoulidis and M. Vakalopoulou, “CAR-MIL: Counterfactual Attention Regularization for Multiple Instance Learning,” arXiv:2609.08419, 2026. Source

[14] J. Pineau, “The Machine Learning Reproducibility Checklist,” version 2.0, Apr. 7, 2020. Source

Appendix Reproducibility Statements

The report and code statements below adapt the Machine Learning Reproducibility Checklist [14]. They distinguish available artifacts from limits in extraction metadata, repeated-run uncertainty and historical timing records.

Table 17. Report reproducibility statement

Report item	Disclosure	Evidence or limitation
Model equations and assumptions	Yes	Sections 2, 4 and 6 define supervision, pooling, masking and the diagnostic.
Algorithmic complexity	Yes	Section 6 gives operation and activation scaling. No sample-complexity guarantee is claimed.
Theoretical performance claims and proofs	Not applicable	No theorem or statistical superiority claim is made.
Dataset statistics and exact partitions	Yes	Section 3 and saved split files; C17 patient overlap is disclosed.
Preprocessing and exclusions	Partial	No added exclusions or sampling; upstream extraction details remain unresolved.
Data access and collection	Yes	Feature source [12]; no new clinical data were collected.
Hyperparameters and selection	Yes	Sections 4 and 7 and the frozen protocol specify settings; no hyperparameter search is reported.
Number of reported real-data runs	Yes	Twelve primary 30-epoch configurations, one seed and one reported test record each. Six new training and test-evaluation runs; baseline and fixed-mixture records are reused. One historical Mac C17 baseline is reported separately. Pilots are excluded from this comparison.
Metric definitions	Yes	Sections 2 and 4; classwise and aggregate results are distinguished.
Variation across repeated runs	No	Only seed 42; no error bars or significance inference.
Runtime and energy	Partial	Section 9 reports wall time for six new runs. Historical timing, peak memory and energy are unavailable.
Full held-out extension results	Yes	Section 7 reports all three datasets with saved probabilities, class metrics and gate diagnostics.

Table 18. Code reproducibility statement

Code item	Disclosure	Evidence or limitation
Dependencies and environment	Yes	requirements.txt and per-run environment.json records.
Training and evaluation programs	Yes	train.py, evaluate_run.py and the research training and evaluation scripts.
Pretrained or learned weights	Yes	Selected baseline, fixed-mixture, mean and adaptive-pooling checkpoints are included.
Configuration and source identity	Yes	Saved configurations, split identifiers, fingerprints and research source snapshots.
Input data	Partial	Large feature files are distributed separately through [12], with provenance hashes.
Metrics, predictions and figures	Partial	Full baseline, adaptive and mean predictions are saved; historical fixed-mixture controls retain aggregate metrics only.
Reproduction instructions and tests	Yes	README and research documentation provide commands and software checks.
Reproduction of every final comparison	Yes	README gives exact training, evaluation and summary commands. Complete configurations, histories, partitions and weights support the twelve primary results.